

Osnove podatkovne znanosti



Fakultet informatike u Puli

Vježba 4

Složeni tipovi podataka u Pythonu

Predavanja: doc. dr. sc. Ivan Lorencin

Vježbe: Elvis Starić mag. inf.

U prethodnim vježbama pojedinačne smo vrijednosti spremali u varijable, provjeravali uvjete i ponavljali obradu pomoću petlji. Takav je pristup dovoljan dok program obrađuje nekoliko unaprijed poznatih vrijednosti. Kada podataka ima više, odvojena varijabla za svaki podatak brzo postaje nepraktična.

U ovoj ćemo vježbi više povezanih vrijednosti organizirati u jednu cjelinu. Obradit ćemo četiri često korištena Pythonova tipa: listu (`list`), n-torku (`tuple`), skup (`set`) i rječnik (`dict`). Naučit ćemo kako se stvaraju, kako se pristupa njihovim elementima, koje se strukture mogu mijenjati te kako odabrati strukturu koja odgovara značenju podataka.

Odabir strukture nije samo pitanje drukčijeg zapisa zagrada. Struktura određuje koje su operacije prirodne, a koje nisu. Ako podatke spremimo u listu, možemo govoriti o prvom i posljednjem elementu. Ako ih spremimo u skup, takvi položaji nemaju smisla, ali zato jednostavno dobivamo jedinstvene vrijednosti. U rječniku podatak ne tražimo prema položaju, nego prema ključu koji opisuje njegovo značenje. Zbog toga ćemo uz sintaksu stalno razmatrati i razlog zbog kojeg određeni tip koristimo.

Novi tipovi neće zamijeniti varijable, petlje i grananja iz prethodnih vježbi. Varijabla će sada često sadržavati cijelu strukturu, petlja će prolaziti kroz njezine elemente, a grananje odlučivati koje elemente treba prihvatiti, preskočiti ili promijeniti. Upravo povezivanje tih koncepata omogućuje obradu većih skupova podataka bez ponavljanja gotovo jednakog koda.

Napravite mapu [Vježba 4](#) i u nju spremajte primjere i svoja rješenja. Svaki potpuni primjer u skripti može se pokrenuti samostalno. Kod se izvršava na vašem računalu, a očekivani ispis i objašnjenje otvaraju se klikom na strelicu.

1 Zašto podatke povezujemo u strukture

1.1 Problem s velikim brojem pojedinačnih varijabli

Pretpostavimo da želimo spremiti bodove pet studenata, ispisati ih i izračunati prosjek. Jedan mogući pristup bio bi spremiti svaki rezultat u zasebnu varijablu:

```
bodovi_1 = 72
bodovi_2 = 88
bodovi_3 = 65
bodovi_4 = 91
bodovi_5 = 76

zbroj = bodovi_1 + bodovi_2 + bodovi_3 + bodovi_4 + bodovi_5
prosjek = zbroj / 5

print("Prosjek:", prosjek)
```

Program radi, ali je broj podataka ugrađen u njegov zapis. Za šestog studenta trebali bismo dodati novu varijablu, proširiti izračun zbroja i promijeniti broj kojim dijelimo. Petlja nam ovdje ne pomaže dovoljno jer pet odvojenih naziva varijabli ne čini jednu cjelinu kroz koju petlja može prolaziti.

Problem nije samo u količini pisanja. Pet varijabli ne prenosi Pythonu informaciju da tih pet vrijednosti pripada istoj skupini. Njihovi slični nazivi korisni su osobi koja čita program, ali Python ih i dalje vidi kao pet odvojenih varijabli. Ne možemo jednim pozivom provjeriti koliko ih ima, sortirati ih ili petljom obraditi svaku od njih. Svaka promjena broja studenata zahtijevala bi promjenu samog programa.

Potreban nam je način da povezane vrijednosti spremimo kao jednu cjelinu. Takva cjelina programu treba omogućiti da podatke promatra kao skupinu, odredi koliko ih ima i istim postupkom obradi svaki podatak. U Pythonu za to koristimo kontejnerske tipove podataka.

Kontejner i element

Kontejner je vrijednost koja u sebi organizira druge vrijednosti. Pojedinačna vrijednost spremljena u kontejner naziva se **element**.

Lista, n-torka, skup i rječnik kontejnerski su tipovi podataka. Svaki od njih povezuje više vrijednosti, ali ih organizira prema drukčijim pravilima. Ta ćemo pravila uvoditi postupno na konkretnim problemima.

Počet ćemo s listom jer ona izravno rješava problem spremanja i obrade više rezultata iz početnog primjera.

2 Liste

Lista je najčešći prvi izbor kada imamo više vrijednosti iste uloge. Umjesto da za svako novo mjerenje mijenjamo program, mjerenja dodajemo u listu, a isti postupak obrade primjenjujemo na sve elemente. Redoslijed je sačuvan, pa možemo razlikovati prvo mjerenje od posljednjega, a promjenjivost liste omogućuje naknadne ispravke, dodavanja i uklanjanja.

U ovoj cjelini najprije ćemo se vratiti na početni problem s bodovima i vidjeti kako lista mijenja organizaciju programa. Zatim ćemo promatrati listu kao spremljeni niz podataka: kako je stvaramo, koliko elemenata sadrži i kako dohvaćamo određeni dio. U sljedećim ćemo cjelinama listu mijenjati i obrađivati petljama. Ta je podjela važna jer „imati više vrijednosti” nije isto što i „znati kako ih pronaći, promijeniti i analizirati”.

2.1 Povezivanje vrijednosti u listu

U početnom primjeru svaki je rezultat bio spremljen u zasebnu varijablu. Zbog toga program nije mogao jednim postupkom proći kroz sve rezultate. Kada ih spremimo u listu, varijabla `bodovi` predstavlja jednu cjelinu kroz koju petlja može prolaziti:

```
bodovi = [72, 88, 65, 91, 76]

zbroj = 0

for rezultat in bodovi:
    zbroj += rezultat

prosijek = zbroj / len(bodovi)
print("Prosijek:", prosijek)
```

▼ Prikaži ispis i objašnjenje

Prosijek: 78.4

Varijabla `bodovi` sada sadrži listu s pet elemenata. Petlja ne mora unaprijed znati koliko rezultata postoji: prolazi kroz sve vrijednosti koje se nalaze u listi. Funkcija `len()` vraća njihov broj, pa se izračun automatski prilagođava ako listi kasnije dodamo ili uklonimo rezultat.

Važno je razlikovati listu od njezinih elemenata. `bodovi` je jedna varijabla čija je vrijednost cijela lista. Broj `72` jedan je element te liste. Varijabla `rezultat` u svakoj iteraciji privremeno predstavlja jedan element: najprije `72`, zatim `88` i tako dalje. Nakon posljednjeg elementa petlja završava bez obzira na to sadrži li lista pet, pedeset ili pet tisuća vrijednosti.

Ovaj primjer pokazuje korist liste prije nego što uvedemo sve pojedinosti njezina zapisa. U nastavku ćemo korak po korak objasniti kako se lista stvara i kako se njezinim elementima pristupa.

2.2 Stvaranje liste i njezini elementi

Lista je prikladna kada imamo više vrijednosti koje čine uređeni niz i očekujemo da ćemo ih tijekom rada možda mijenjati. To mogu biti bodovi studenata, dnevne temperature, nazivi datoteka ili redoslijed koraka nekog postupka.

Riječ *uređeni* ovdje znači da Python pamti slijed u kojem su elementi zapisani ili dodani. U listi temperatura `[18.5, 20.0, 19.5]` vrijednost `18.5` nalazi se prije `20.0`. To ne znači da je lista automatski sortirana, nego samo da svaki element ima stabilan položaj. Jednake se vrijednosti smiju pojaviti više puta jer dva dana mogu imati jednaku temperaturu ili dva studenta jednak broj bodova.

Listu zapisujemo uglatim zagradama. Elemente razdvajamo zarezima:

```
temperature = [18.5, 20.0, 19.5, 22.0]
gradovi = ["Pula", "Rijeka", "Pazin"]
prazna_lista = []

print(temperature)
print(gradovi)
print(prazna_lista)
print(type(temperature))
```

▼ Prikaži očekivani ispis

```
[18.5, 20.0, 19.5, 22.0]
['Pula', 'Rijeka', 'Pazin']
[]
<class 'list'>
```

Prazna lista postoji i ima nula elemenata. U prikazu tekstualnih elemenata Python dodaje navodnike kako bi bilo vidljivo da je riječ o vrijednostima tipa `str`.

Prazna lista korisna je kada na početku još nemamo rezultate, ali ih namjeravamo prikupljati tijekom izvođenja programa. Umjesto stvaranja posebne varijable tek nakon prvog unosa, listu možemo napraviti unaprijed i zatim joj dodavati svaki prihvaćeni podatak. `[]` zato nije „nedostajući podatak“, nego postojeća lista koja trenutačno nema elemenata.

Python dopušta da ista lista sadrži vrijednosti različitih tipova:

```
razlicite_vrijednosti = ["Ana", 21, 4.5, True]
```

Takav je zapis valjan, ali nije uvijek dobra organizacija podataka. Kada elementi predstavljaju istu pojavu, dosljedan tip olakšava obradu. Primjerice, listu bodova lakše je zbrajati ako su svi elementi brojevi. Različite podatke o jednoj osobi kasnije ćemo smislenije organizirati n-torkom ili rječnikom.

💡 Lista

Lista (`list`) uređena je i promjenjiva zbirka elemenata. Čuva redoslijed elemenata, dopušta ponavljanje jednakih vrijednosti i može sadržavati vrijednosti različitih tipova.

2.3 Broj elemenata i provjera pripadnosti

Nakon spremanja podataka često prvo želimo odgovoriti na dva pitanja: koliko ih ima i pojavljuje li se određena vrijednost među njima. Bez ugrađenih operacija morali bismo pokrenuti petlju, ručno održavati brojač i posebno uspoređivati svaki element. Python za ta česta pitanja nudi izravne zapise.

Funkcija `len()` nad listom vraća broj njezinih elemenata, a ne zbroj ili veličinu pojedinačnih vrijednosti. Lista od tri dugačka teksta i lista od tri jednoznamenakasta broja obje imaju duljinu `3`:

```
gradovi = ["Pula", "Rijeka", "Pazin"]

print("Broj gradova:", len(gradovi))
print("Pula" in gradovi)
print("Zagreb" not in gradovi)
```

▼ Prikaži ispis i objašnjenje

```
Broj gradova: 3
True
True
```

Operator `in` provjerava postoji li vrijednost među elementima liste. Ne pretražuje dijelove tekstualnih elemenata: izraz `"Pul" in gradovi` dao bi `False` jer lista ne sadrži element koji je točno jednak tekstu `"Pul"`.

Provjera pripadnosti korisna je prije uklanjanja ili dodavanja vrijednosti. Primjerice, naziv možemo dodati samo ako već ne postoji:

```
gradovi = ["Pula", "Rijeka", "Pazin"]
novi_grad = "Pula"

if novi_grad not in gradovi:
    gradovi.append(novi_grad)

print(gradovi)
```

Lista ostaje nepromijenjena jer se `"Pula"` već nalazi među elementima.

U ovom primjeru `in` ne govori na kojem se položaju vrijednost nalazi ni koliko se puta pojavljuje. Rezultat je samo `True` ili `False`. To je dovoljno kada odluka programa ovisi o postojanju vrijednosti, primjerice kada želimo spriječiti dodavanje duplikata ili provjeriti je li unesena oznaka dopuštena.

2.4 Pristup elementu pomoću indeksa

Petlja je prikladna kada želimo obraditi sve elemente, ali ponekad nas zanima točno određeni položaj: prvi zabilježeni rezultat, posljednja unesena vrijednost ili treći korak postupka. Budući da lista čuva redoslijed, svakom elementu možemo pristupiti pomoću njegova indeksa.

Kao kod teksta, indeksi počinju od nule. Prvi element ima indeks `0`, drugi indeks `1`, a posljednji nenegativni indeks jednak je `len(lista) - 1`. Indeks je broj položaja, a rezultat indeksiranja stvarna vrijednost spremljena na tom položaju. Ako je `predmeti[0]` tekst `"Matematika"`, rezultat izraza tipa je `str`, a ne `list`.

```
predmeti = ["Matematika", "Programiranje", "Podatkovna znanost"]

print(predmeti[0])
print(predmeti[1])
print(predmeti[-1])
```

▼ Prikaži očekivani ispis

```
Matematika
Programiranje
Podatkovna znanost
```

Negativni indeks `-1` označava posljednji element, `-2` pretposljednji i tako dalje.

Negativni su indeksi osobito korisni kada ne znamo duljinu liste. Posljednjem elementu možemo pristupiti izrazom `predmeti[len(predmeti) - 1]`, ali `predmeti[-1]` kraće i jasnije izražava istu namjeru.

Pozitivni i negativni indeks ne stvaraju kopiju elementa, oba samo pronalaze vrijednost na određenom položaju.

⚠ Pripazite na granice indeksa

Za listu s tri elementa postoje indeksi `0`, `1` i `2`. Pokušaj pristupa elementu `predmeti[3]` završava pogreškom `IndexError`. Prazna lista nema ni prvi ni posljednji element, pa nad njom ne možemo koristiti ni indeks `0` ni indeks `-1`.

2.5 Izdvajanje dijela liste

Indeksiranje vraća jedan element. Kada trebamo nekoliko uzastopnih elemenata, primjerice rezultate prvih triju dana ili posljednja dva mjerenja, ponavljanje pojedinačnih indeksa postaje nespretno. Za izdvajanje uzastopnog dijela koristimo rezanje.

Rezanjem liste dobivamo novu listu s odabranim rasponom elemenata. Zapis `lista[pocetak:kraj]` uključuje element na početnom indeksu, ali ne uključuje element na završnom indeksu. Isključena završna granica odgovara ponašanju funkcije `range()` i omogućuje da broj elemenata raspona izračunamo kao `kraj - pocetak`.

```
bodovi = [48, 61, 73, 85, 92, 67]
```

```
print(bodovi[1:4])
print(bodovi[:3])
print(bodovi[3:])
print(bodovi[-2:])
```

▼ Prikaži ispis i granice raspona

```
[61, 73, 85]
[48, 61, 73]
[85, 92, 67]
[92, 67]
```

Raspon `[1:4]` obuhvaća indekse `1`, `2` i `3`. Izostavljena početna granica znači početak liste, a izostavljena završna granica kraj liste. Rezanje izvan postojećih granica ne uzrokuje `IndexError`, Python vraća dio koji postoji.

Rezultat rezanja uvijek je lista, čak i kada obuhvaća jedan element ili nijedan element. `bodovi[2:3]` daje listu s jednim elementom, dok `bodovi[2]` daje sam broj. Ta razlika utječe na kasnije operacije: broj možemo izravno usporediti s drugim brojem, a listu dalje obrađujemo kao zbirku.

Pregled novih zapisa, funkcija i operatora

Zapis, funkcija ili operator	Namjena
<code>[a, b, c]</code>	Stvara listu s navedenim elementima
<code>[]</code>	Stvara praznu listu
<code>len(lista)</code>	Vraća broj elemenata liste
<code>vrijednost in lista</code>	Provjerava nalazi li se vrijednost u listi
<code>vrijednost not in lista</code>	Provjerava da se vrijednost ne nalazi u listi
<code>lista[indeks]</code>	Dohvaća jedan element prema položaju
<code>lista[pocetak:kraj]</code>	Stvara novu listu s odabranim rasponom elemenata

Zadatak

Zadana je lista dnevnih brojeva posjeta `[125, 148, 132, 171, 160, 154, 139]`. Ispišite broj elemenata, prvi i posljednji rezultat, rezultate od drugog do četvrtog dana te provjerite nalazi li se vrijednost `171` u listi.

▼ Prikaži moguće rješenje i ispis

```
posjeti = [125, 148, 132, 171, 160, 154, 139]

print("Broj dana:", len(posjeti))
print("Prvi dan:", posjeti[0])
print("Posljednji dan:", posjeti[-1])
print("Drugi do četvrti dan:", posjeti[1:4])
print("Postoji 171:", 171 in posjeti)
```

Broj dana: 7

Prvi dan: 125

Posljednji dan: 139

Drugi do četvrti dan: [148, 132, 171]

Postoji 171: True

3 Mijenjanje liste

Podaci u programu često nisu konačni u trenutku stvaranja liste. Može stići novo mjerenje, korisnik može otkazati prijavu ili možemo otkriti pogrešan unos. Kada bismo za svaku takvu promjenu ponovno pisali cijelu listu, morali bismo ručno prepisivati sve nepromijenjene elemente i povećali bismo mogućnost nove pogreške.

Promjenjivost liste omogućuje da izmijenimo samo potreban dio postojeće strukture. Pritom treba razlikovati tri namjere: zamijeniti element koji već postoji, dodati novi element ili ukloniti element. Python za njih koristi različite zapise jer svaka operacija drukčije utječe na duljinu i položaje u listi.

3.1 Promjena postojećeg elementa

Tekst tipa `str` nismo mogli promijeniti zamjenom znaka na određenom indeksu. Lista je drukčija: njezine elemente možemo zamijeniti nakon stvaranja.

Zamjena je prikladna kada podatak i dalje pripada istom položaju, ali mu je vrijednost pogrešna ili zastarjela. Duljina liste tada ostaje jednaka. Elementi prije i poslije izmijenjenog položaja ne pomiču se, pa ostali indeksi zadržavaju isto značenje.

Pretpostavimo da je treći rezultat pogrešno unesen kao `54`, a treba biti `74`:

```
bodovi = [68, 81, 54, 90]

bodovi[2] = 74

print(bodovi)
```

▼ Prikaži ispis i objašnjenje

```
[68, 81, 74, 90]
```

Indeks `2` označava treći element. Dodjela ne stvara novu listu, nego mijenja postojeću vrijednost na tom položaju.

Lijeva strana `bodovi[2]` određuje mjesto promjene, a desna strana `74` novu vrijednost. Prije dodjele Python mora pronaći postojeći položaj, pa indeks i ovdje mora biti unutar granica liste. Ne možemo dodati peti element dodjelom `bodovi[4] = ...` ako lista trenutačno ima samo četiri elementa; za dodavanje koristimo metode opisane u nastavku.

💡 Promjenjivost

Promjenjiv tip dopušta promjenu sadržaja postojeće vrijednosti nakon njezina stvaranja. Lista je promjenjiva: možemo zamijeniti, dodati ili ukloniti njezine elemente.

3.2 Dodavanje elemenata

Zamislamo program koji tijekom prijave ne zna unaprijed koliko će osoba sudjelovati. Prazna ili djelomično popunjena lista mora rasti nakon svakog valjanog unosa. Za to ne koristimo dodjelu prema nepostojećem indeksu, nego metode za dodavanje.

Metoda `append()` dodaje jednu vrijednost na kraj liste. Duljina se povećava za jedan, a postojeći elementi zadržavaju svoje položaje:

```
prijavljeni = ["Ana", "Ivo"]  
  
prijavljeni.append("Mia")  
  
print(prijavljeni)
```

```
['Ana', 'Ivo', 'Mia']
```

Kada želimo dodati više elemenata iz druge zbirke, koristimo `extend()`:

```
prijavljeni = ["Ana", "Ivo"]  
novi_prijavljeni = ["Mia", "Luka"]  
  
prijavljeni.extend(novi_prijavljeni)  
  
print(prijavljeni)
```

```
['Ana', 'Ivo', 'Mia', 'Luka']
```

Razlika je važna. `append(novi_prijavljeni)` dodao bi cijelu drugu listu kao jedan element, dok `extend(novi_prijavljeni)` dodaje njezine elemente pojedinačno.

Nakon `append(novi_prijavljeni)` duljina početne liste povećala bi se za jedan, a posljednji element bio bi lista `["Mia", "Luka"]`. Nakon `extend(novi_prijavljeni)` duljina se povećava za dva i oba su imena elementi iste razine. Izbor zato ovisi o tome želimo li spremiti cijelu zbirku kao jedan podatak ili spojiti njezine elemente s postojećima.

Metoda `insert(indeks, vrijednost)` umeće vrijednost na određeni položaj i pomiče kasnije elemente:

```
redoslijed = ["Ana", "Mia", "Luka"]  
redoslijed.insert(1, "Ivo")  
  
print(redoslijed)
```

```
['Ana', 'Ivo', 'Mia', 'Luka']
```

`insert()` koristimo samo kada novi element pripada određenom mjestu u redoslijedu. Umetanje na početak pomiče sve postojeće elemente za jedno mjesto. Ako položaj nije važan, `append()` jasnije pokazuje da samo proširujemo popis i obično je prirodniji izbor.

3.3 Uklanjanje elemenata

Kod uklanjanja prvo trebamo znati što imamo: samu vrijednost koju želimo ukloniti ili njezin položaj. Te dvije informacije nisu isto. Ako korisnik otkaže prijavu osobe "Ana", poznajemo vrijednost. Ako obrađujemo posljednji zadatak u redu čekanja, važan nam je položaj.

Metoda `remove()` prima vrijednost i uklanja njezino prvo pojavljivanje:

```
oznake = ["A", "B", "A", "C"]
oznake.remove("A")

print(oznake)
```

```
['B', 'A', 'C']
```

Druga vrijednost "A" ostaje u listi. Ako tražena vrijednost ne postoji, `remove()` uzrokuje `ValueError`, pa je prije uklanjanja često korisno provjeriti pripadnost operatorom `in`.

Metoda `pop()` uklanja element prema indeksu i vraća uklonjenu vrijednost. Bez argumenta uklanja posljednji element:

```
zadaci = ["unos", "provjera", "izvještaj"]

zavrsni_zadatak = zadaci.pop()
prvi_zadatak = zadaci.pop(0)

print("Uklonjeno s kraja:", zavrsni_zadatak)
print("Uklonjeno s početka:", prvi_zadatak)
print("Preostalo:", zadaci)
```

▼ Prikaži očekivani ispis

```
Uklonjeno s kraja: izvještaj
Uklonjeno s početka: unos
Preostalo: ['provjera']
```

Vraćena vrijednost korisna je kada uklonjeni element želimo dalje obraditi. Pogrešan indeks uzrokuje `IndexError`, a poziv `pop()` nad praznom listom također nije moguć.

Razlika između `remove()` i `pop()` nije samo u argumentu. `remove()` koristimo zbog sadržaja elementa i ne vraća uklonjenu vrijednost. `pop()` koristimo zbog položaja, a uklonjenu vrijednost možemo spremiti u varijablu. Ako nam uklonjeni element više nije potreban, rezultat metode `pop()` ne moramo spremiti.

Metoda `clear()` uklanja sve elemente, ali sama lista i dalje postoji:

```
vrijednosti = [1, 2, 3]
vrijednosti.clear()

print(vrijednosti)
print(len(vrijednosti))
```

```
[]
```

```
0
```

To nije isto što i brisanje varijable. Nakon `clear()` možemo ponovno koristiti istu listu i dodavati joj nove elemente. Operacija je korisna kada želimo započeti novo prikupljanje podataka, a ostatak programa i dalje očekuje da varijabla s listom postoji.

3.4 Dvije varijable mogu upućivati na istu listu

Kod brojeva smo vrijednost jedne varijable mogli dodijeliti drugoj i poslije ih mijenjati neovisno. Kod lista je važno razlikovati novu varijablu od nove liste.

Ova se razlika često pojavi kada želimo sačuvati izvorne podatke i napraviti radnu verziju za sortiranje ili ispravke. Naziv varijable nije sama lista možemo ga promatrati kao oznaku koja vodi do objekta u memoriji. Dodjela drugoj varijabli kopira tu vezu, a ne sve elemente u novi objekt.

```
izvorni_bodovi = [70, 80, 90]
kopija = izvorni_bodovi

kopija[0] = 100

print("Izvorni bodovi:", izvorni_bodovi)
print("Kopija:", kopija)
```

▼ Prikaži ispis i objašnjenje

```
Izvorni bodovi: [100, 80, 90]
Kopija: [100, 80, 90]
```

Dodjela `kopija = izvorni_bodovi` nije stvorila novu listu. Obje varijable upućuju na isti promjenjivi objekt, pa je promjena vidljiva kroz oba naziva.

To ponašanje nije pogreška Pythona. Ponekad namjerno želimo da dva dijela programa rade s istom listom. Problem nastaje kada očekujemo neovisnu kopiju, a napravili smo samo drugi naziv. Tada promjena „kopije” neočekivano mijenja i podatke koje smo namjeravali sačuvati.

Ako želimo zasebnu listu s istim elementima, možemo koristiti metodu `copy()`:

```
izvorni_bodovi = [70, 80, 90]
kopija = izvorni_bodovi.copy()

kopija[0] = 100

print("Izvorni bodovi:", izvorni_bodovi)
print("Kopija:", kopija)
```

Izvorni bodovi: [70, 80, 90]
Kopija: [100, 80, 90]

⚠ Dodjela nije kopiranje liste

Zapis `druga = prva` daje istoj listi još jedan naziv. Za zasebnu listu koristite `prva.copy()`. Kod ugniježđenih struktura ova kopija odvaja samo vanjsku listu; dublje kopiranje obradit ćemo samo ako nam bude potrebno.

🔔 Pregled novih zapisa i metoda

Zapis ili metoda	Namjena
<code>lista[indeks] = vrijednost</code>	Mijenja element na zadanom položaju
<code>append(vrijednost)</code>	Dodaje jedan element na kraj liste
<code>extend(zbirka)</code>	Dodaje sve elemente druge zbirke na kraj liste
<code>insert(indeks, vrijednost)</code>	Umeće element na zadani položaj
<code>remove(vrijednost)</code>	Uklanja prvo pojavljivanje zadane vrijednosti
<code>pop(indeks)</code>	Uklanja i vraća element; bez indeksa uklanja posljednji
<code>clear()</code>	Uklanja sve elemente liste
<code>copy()</code>	Stvara novu listu s istim elementima

📖 Zadatak

Zadana je lista prijavljenih studenata `['Ana', 'Ivo', 'Mia']`. Napravite njezinu neovisnu kopiju koja će služiti kao radna lista. U radnu listu dodajte `Luka` na kraj, umetnite `Ena` na drugo mjesto, uklonite `Ivo` prema vrijednosti te spremite i ispišite posljednju osobu uklonjenu metodom `pop()`. Na kraju ispišite izvornu listu, preostalu radnu listu i broj njezinih elemenata. Izvorna lista mora ostati nepromijenjena.

▼ Prikaži moguće rješenje i ispis

```
prijavljeni = ["Ana", "Ivo", "Mia"]
radna_lista = prijavljeni.copy()

radna_lista.append("Luka")
radna_lista.insert(1, "Ena")
radna_lista.remove("Ivo")
uklonjena_osoba = radna_lista.pop()

print("Uklonjena osoba:", uklonjena_osoba)
print("Izvorna lista:", prijavljeni)
print("Radna lista:", radna_lista)
print("Broj preostalih:", len(radna_lista))
```

Uklonjena osoba: Luka
Izvorna lista: ['Ana', 'Ivo', 'Mia']
Radna lista: ['Ana', 'Ena', 'Mia']
Broj preostalih: 3

4 Obrada podataka u listi

Samo spremanje vrijednosti nije krajnji cilj obrade podataka. Iz liste želimo napraviti izvještaj, pronaći izdvojene vrijednosti, izračunati sažetak ili stvoriti novu listu prema zadanom pravilu. Sve te aktivnosti povezuju strukture iz ove vježbe s petljama i grananjima iz prethodne.

Prije pisanja koda korisno je odrediti obrađujemo li svaki element jednako ili samo neke elemente. Ako svaki element sudjeluje u zbroju, dovoljna je petlja ili ugrađena funkcija. Ako element treba zadovoljiti uvjet, u petlju dodajemo grananje. Ako rezultat obrade treba ostati dostupan nakon petlje, pripremamo akumulator, brojač ili novu listu prije početka ponavljanja.

4.1 Prolazak kroz elemente

Bez petlje bismo za svaku temperaturu morali napisati zaseban `print()`. Takav bi program ovisio o točnom broju mjerenja i prestao bi biti potpun čim listi dodamo novi element. Petlja povezuje jednu zapisanu obradu sa svim elementima zbirke.

Kada želimo obraditi svaki element liste, petlja `for` izravno preuzima vrijednosti redom. Ne trebamo unaprijed znati broj elemenata ni ručno mijenjati indeks. Izvor petlje je lista, a varijabla petlje u svakoj iteraciji dobiva sljedeću vrijednost.

```
temperature = [18.5, 20.0, 19.5, 22.0]

for temperatura in temperature:
    print(f"Izmjereno: {temperatura:.1f} °C")
```

▼ Prikaži očekivani ispis

```
Izmjereno: 18.5 °C
Izmjereno: 20.0 °C
Izmjereno: 19.5 °C
Izmjereno: 22.0 °C
```

U svakoj iteraciji varijabla `temperatura` dobiva sljedeći element liste. Promjena te varijable sama po sebi ne mijenja element liste, za promjenu na određenom položaju potreban nam je indeks.

Ako napišemo `temperatura = 0` unutar petlje, samo mijenjamo vrijednost varijable petlje u toj iteraciji. Lista ostaje jednaka jer nismo odredili koji njezin položaj treba promijeniti. Kada je cilj samo čitanje i računanje, vrijednosti su dovoljne. Kada trebamo mijenjati elemente na njihovim mjestima, potreban nam je i indeks.

Ponekad su nam potrebni i položaj i vrijednost. Mogli bismo prolaziti indeksima pomoću `range(len(lista))`, ali funkcija `enumerate()` jasnije izražava tu namjeru:

```
gradovi = ["Pula", "Rijeka", "Pazin"]

for indeks, grad in enumerate(gradovi):
    print(indeks, grad)
```

```
0 Pula
1 Rijeka
2 Pazin
```

Funkcija `enumerate()` za svaki element daje dvije vrijednosti: njegov indeks i sam element. Ako želimo numeriranje za korisnika od jedan, možemo zadati početak:

Indeks i redni broj ne moraju biti jednaki. Indeks pripada Pythonovu načinu pristupa listi i počinje od nule. Redni broj u izvještaju često počinje od jedan jer je tako prirodnije čitatelju. Argument `start=1` mijenja broj koji daje `enumerate()`, ali ne mijenja stvarne indekse liste.

```
for redni_broj, grad in enumerate(gradovi, start=1):
    print(f"{redni_broj}. {grad}")
```

```
1. Pula
2. Rijeka
3. Pazin
```

4.2 Zbroj, najmanja i najveća vrijednost

Zbroj bismo mogli ponovno izračunati akumulatorom kao u prethodnoj vježbi. Taj je postupak važan kada izračun ima posebno pravilo, ali za običan zbroj svih elemenata nepotrebno ponavlja već ugrađenu mogućnost jezika.

Za listu brojeva Python ima funkcije koje obavljaju česte izračune. `sum()` vraća zbroj, `min()` najmanju, a `max()` najveću vrijednost. Funkcije primaju cijelu zbirku i vraćaju jednu sažetu vrijednost:

```
bodovi = [72, 88, 65, 91, 76]

print("Zbroj:", sum(bodovi))
print("Najmanje:", min(bodovi))
print("Najviše:", max(bodovi))
print("Prosjek:", sum(bodovi) / len(bodovi))
```

▼ Prikaži ispis i napomenu o praznoj listi

```
Zbroj: 392
Najmanje: 65
Najviše: 91
Prosjek: 78.4
```

`sum([])` vraća `0`, ali `min([])` i `max([])` nemaju rezultat jer prazna lista nema najmanji ni najveći element. Izračun prosjeka prazne liste pokušao bi dijeliti nulom. Zato prije tih operacija provjeravamo ima li lista elemenata.

Python nema zasebnu ugrađenu funkciju `average()` u osnovnom jeziku. Prosjek dobivamo dijeljenjem zbroja s brojem elemenata. Provjera prazne liste dio je logike našeg programa: moramo odlučiti treba li ispisati poruku, preskočiti izračun ili upotrijebiti neku dogovorenu zamjensku vrijednost.

```
bodovi = []

if len(bodovi) > 0:
    prosjek = sum(bodovi) / len(bodovi)
    print("Prosjek:", prosjek)
else:
    print("Nema rezultata za izračun prosjeka.")
```

4.3 Brojanje elemenata i pronalaženje položaja

Pitanja „koliko puta?” i „gdje prvi put?” zahtijevaju različite odgovore. Ako analiziramo ocjene, možda nas zanima broj petica. Ako želimo pronaći položaj prvog zapisa s peticom, potreban nam je indeks.

Metoda `count()` vraća koliko se puta vrijednost pojavljuje u listi. Metoda `index()` vraća indeks prvog pojavljivanja. Obje uspoređuju traženu vrijednost s elementima, ali rezultat i ponašanje kada vrijednost ne postoji nisu jednaki:

```
ocjene = [4, 5, 3, 5, 4, 5]

print("Broj petica:", ocjene.count(5))
print("Prva petica ima indeks:", ocjene.index(5))
```

Broj petica: 3

Prva petica ima indeks: 1

Ako vrijednost ne postoji, `count()` vraća `0`, a `index()` uzrokuje `ValueError`. Kada nismo sigurni postoji li vrijednost, prije poziva `index()` možemo koristiti operator `in`.

`index()` ne vraća sve položaje ako se vrijednost ponavlja. Za listu `[4, 5, 3, 5]` rezultat `index(5)` je `1`, iako se druga petica nalazi na indeksu `3`. Ako trebamo sve položaje, prolazimo listom pomoću `enumerate()` i spremamo indekse koji zadovoljavaju uvjet.

4.4 Izdvajanje elemenata koji zadovoljavaju uvjet

Pretpostavimo da iz svih rezultata želimo izdvojiti samo prolazne, odnosno one s najmanje `60` bodova. Izvorna lista treba ostati sačuvana, pa stvaramo novu praznu listu i u nju dodajemo samo odgovarajuće vrijednosti:

To je drukčiji zadatak od brojanja prolaznih rezultata. Brojač bi nam rekao koliko ih ima, ali nakon petlje ne bismo imali same vrijednosti. Nova lista čuva odabrane elemente i omogućuje da ih poslije ispišemo, sortiramo ili dodatno analiziramo.

```
bodovi = [48, 61, 73, 55, 85, 92]
prolazni = []

for rezultat in bodovi:
    if rezultat >= 60:
        prolazni.append(rezultat)

print("Svi rezultati:", bodovi)
print("Prolazni rezultati:", prolazni)
```

▼ Prikaži ispis i tijek obrade

Svi rezultati: [48, 61, 73, 55, 85, 92]
Prolazni rezultati: [61, 73, 85, 92]

Nova lista na početku nema elemenata. Svaka vrijednost prolazi provjeru, a `append()` se izvršava samo kada je uvjet istinit. Izvorna lista nije promijenjena.

Za rezultat `48` uvjet je lažan i lista `prolazni` ostaje prazna. Za `61` uvjet je istinit pa se vrijednost dodaje. Isti se postupak ponavlja za svaki element. Nakon petlje nova lista sadrži samo vrijednosti koje su u trenutku provjere zadovoljile pravilo.

Ovakav postupak naziva se filtriranje podataka. Python ima i kraći zapis koji se zove list comprehension, ali u ovoj vježbi koristimo eksplicitnu petlju i grananje jer jasno prikazuju svaki korak obrade.

4.5 Sortiranje liste

Redoslijed prikupljanja i redoslijed prikaza nisu uvijek isti. Rezultate možda čuvamo redom kojim su zaprimljeni, ali ih u izvještaju želimo prikazati od najmanjeg prema najvećem. Prije sortiranja zato odlučujemo smijemo li izgubiti izvorni redoslijed.

Metoda `sort()` mijenja postojeću listu. Nakon poziva stari redoslijed više nije dostupan kroz tu varijablu, osim ako smo prethodno napravili kopiju:

```
bodovi = [72, 88, 65, 91, 76]

bodovi.sort()
print(bodovi)

bodovi.sort(reverse=True)
print(bodovi)
```

[65, 72, 76, 88, 91]
[91, 88, 76, 72, 65]

Ako izvorni redoslijed želimo sačuvati, koristimo funkciju `sorted()`, koja vraća novu listu:

```
bodovi = [72, 88, 65, 91, 76]
sortirani = sorted(bodovi)

print("Izvorno:", bodovi)
print("Sortirano:", sortirani)
```

Izvorno: [72, 88, 65, 91, 76]
Sortirano: [65, 72, 76, 88, 91]

Funkcija `sorted()` može primiti i druge zbirke, primjerice n-torku ili skup, ali rezultat je uvijek nova lista. Argument `reverse=True` obrće smjer sortiranja. Brojevi se uspoređuju prema veličini, a tekstovi abecednim redom prema pravilima usporedbe znakova, prije sortiranja zato elementi moraju biti međusobno usporedivi.

⚠ `sort()` ne vraća sortiranu listu

Metoda `sort()` mijenja postojeću listu i vraća `None`. Zapis `sortirani = bodovi.sort()` zato ne sprema sortiranu listu u varijablu `sortirani`. Upotrijebite `bodovi.sort()` ako želite promijeniti listu ili `sortirani = sorted(bodovi)` ako želite novu listu.

🔔 Pregled novih funkcija i metoda

Funkcija ili metoda	Namjena
<code>enumerate(zbirka, start=0)</code>	Daje indeks ili redni broj zajedno s elementom
<code>sum(zbirka)</code>	Vraća zbroj brojčanih elemenata
<code>min(zbirka)</code>	Vraća najmanji element neprazne zbirke
<code>max(zbirka)</code>	Vraća najveći element neprazne zbirke
<code>count(vrijednost)</code>	Vraća broj pojavljivanja vrijednosti
<code>index(vrijednost)</code>	Vraća indeks prvog pojavljivanja vrijednosti
<code>sort(reverse=False)</code>	Sortira postojeću listu
<code>sorted(zbirka, reverse=False)</code>	Vraća novu sortiranu listu

📄 Zadatak

Zadana je lista vremena rješavanja zadatka u minutama `[42, 35, 51, 29, 35, 47]`. Ispišite svako vrijeme uz redni broj od jedan, izračunajte prosjek, ispišite najkraće i najdulje vrijeme, prebrojite koliko se puta pojavljuje `35` te napravite novu sortiranu listu bez promjene izvornog redoslijeda.

▼ Prikaži moguće rješenje i ispis

```
vremena = [42, 35, 51, 29, 35, 47]

for redni_broj, vrijeme in enumerate(vremena, start=1):
    print(f"{redni_broj}. mjerenje: {vrijeme} min")

prosjek = sum(vremena) / len(vremena)
sortirana_vremena = sorted(vremena)

print("Prosjek:", prosjek)
print("Najkraće:", min(vremena))
print("Najdulje:", max(vremena))
print("Broj rezultata 35:", vremena.count(35))
print("Izvorno:", vremena)
print("Sortirano:", sortirana_vremena)
```

```
1. mjerenje: 42 min
2. mjerenje: 35 min
3. mjerenje: 51 min
4. mjerenje: 29 min
5. mjerenje: 35 min
6. mjerenje: 47 min
Prosjek: 39.833333333333336
Najkraće: 29
Najdulje: 51
Broj rezultata 35: 2
Izvorno: [42, 35, 51, 29, 35, 47]
Sortirano: [29, 35, 35, 42, 47, 51]
```

5 N-torke

Lista i n-torka na prvi pogled izgledaju vrlo slično: obje čuvaju uređene elemente, imaju indekse i dopuštaju prolazak petljom. Razlika postaje važna kada razmišljamo o tome smije li se sam zapis mijenjati. Lista često predstavlja skup podataka koji raste ili se uređuje, dok n-torka često predstavlja jednu dovršenu cjelinu s unaprijed poznatim položajima.

Nepromjenjivost nije zaštita od svake moguće programske pogreške, ali jasno izražava namjeru. Osoba koja vidi n-torku zna da kod ne bi trebao dodavati nove položaje, uklanjati postojeće niti zamijeniti njihove vrijednosti. Python tu namjeru i provodi tako što takve operacije ne dopušta.

5.1 Kada podatke ne želimo mijenjati

Lista je korisna kada se zbirka tijekom programa mijenja. Ponekad promjena nije poželjna. Koordinate točke, dimenzije slike ili datum zapisan kao godina, mjesec i dan predstavljaju jedan zapis čiji položaji imaju unaprijed određeno značenje.

Kod takvih zapisa promjena broja elemenata često bi promijenila i njihovo značenje. Koordinata s dvije vrijednosti može predstavljati zemljopisnu širinu i dužinu. Dodamo li treću vrijednost bez dogovora što ona znači, više nemamo jednak oblik podatka. N-torka ne objašnjava značenje položaja umjesto nas, ali sprječava slučajno mijenjanje njezina oblika nakon stvaranja.

Ako koordinate spremimo u listu, program dopušta njihovo dodavanje i uklanjanje čak i kada bi takva promjena narušila očekivani oblik zapisa:

```
koordinate = [44.8666, 13.8496]
koordinate.append(100)

print(koordinate)
```

▼ Prikaži očekivani ispis i objašnjenje

```
[44.8666, 13.8496, 100]
```

Metoda `append()` uspješno dodaje treću vrijednost jer je lista promjenjiva. Python ne zna da u ovom programu koordinate trebaju sadržavati točno dvije vrijednosti, pa takvu promjenu ne smatra pogreškom.

Kada želimo naglasiti da zbirka predstavlja cjelovit zapis koji se ne treba mijenjati, možemo koristiti n-torku.

To ne znači da n-torku treba koristiti za svaki podatak koji se trenutačno ne mijenja. Ako je popis samo privremeno nepromijenjen, ali ga program kasnije treba proširiti, lista je i dalje prikladnija. N-torku biramo kada je nepromjenjivost dio značenja zapisa, a ne samo trenutačno stanje programa.

💡 N-torka

N-torka (`tuple`) uređena je i nepromjenjiva zbirka elemenata. Čuva redoslijed, dopušta ponavljanje jednakih vrijednosti i nakon stvaranja ne dopušta zamjenu, dodavanje ni uklanjanje elemenata.

5.2 Stvaranje i pristup elementima

N-torka čuva elemente istim redoslijedom kojim su navedeni. Zato joj možemo pristupiti već poznatim indeksima i koristiti iste obrasce čitanja kao kod liste. N-torku najčešće zapisujemo oblim zagrada:

```
koordinate = (44.8666, 13.8496)
datum = (2026, 10, 15)
prazna_ntorka = ()

print(koordinate)
print(datum[0])
print(datum[-1])
print(len(prazna_ntorka))
```

```
(44.8666, 13.8496)
2026
15
0
```

Indeksi, negativni indeksi, rezanje, `len()`, `in`, `count()` i `index()` ponašaju se kao kod liste. Bitna je razlika u promjenjivosti:

Rezanje n-torke vraća novu n-torku, a ne listu. Primjerice, `datum[:2]` daje `(2026, 10)`. Operacije koje samo čitaju sadržaj dostupne su jer ne mijenjaju izvorni zapis. Nisu dostupne metode poput `append()`, `remove()` i `sort()` jer bi one promijenile broj, vrijednost ili raspored elemenata.

```
koordinate = (44.8666, 13.8496)
koordinate[0] = 45.0
```

Taj pokušaj završava pogreškom `TypeError` jer n-torka ne podržava promjenu elementa.

Ako ipak trebamo novu verziju zapisa, stvaramo novu n-torku i dodjeljujemo je varijabli. Stara n-torka nije promijenjena, varijabla nakon nove dodjele samo upućuje na novu vrijednost. To je isti princip koji smo vidjeli kod tekstova.

⚠ N-torka s jednim elementom treba zarez

Zapis `(5)` predstavlja broj `5` u zagradama, a ne n-torku. N-torku s jednim elementom zapisujemo kao `(5,)`. Zarez određuje n-torku, oble zagrade prije svega poboljšavaju čitljivost.

```
broj = (5)
jedan_element = (5,)

print(type(broj))
print(type(jedan_element))
```

```
<class 'int'>
<class 'tuple'>
```

5.3 Raspakiravanje n-torke

Izraz `datum[0]`, `datum[1]` i `datum[2]` ispravno dohvaća dijelove datuma, ali pri čitanju koda moramo pamtit i što svaki položaj znači. Kada ćemo iste dijelove koristiti više puta, jasnije ih je imenovati.

Elemente možemo odjednom dodijeliti zasebnim varijablama. Taj postupak nazivamo raspakiravanje. Dodjela se izvodi slijeva nadesno: prvi element odlazi u prvu varijablu, drugi u drugu i treći u treću.

```
datum = (2026, 10, 15)
godina, mjesec, dan = datum

print("Godina:", godina)
print("Mjesec:", mjesec)
print("Dan:", dan)
```

▼ Prikaži ispis i objašnjenje

```
Godina: 2026
Mjesec: 10
Dan: 15
```

Broj varijabli mora odgovarati broju elemenata. Ako n-torka ima tri elementa, a navedemo dvije varijable, Python ne zna gdje smjestiti preostalu vrijednost i javlja `ValueError`.

Raspakiravanje ne povezuje varijable trajno s n-torkom. Nakon dodjele `godina`, `mjesec` i `dan` samostalne su varijable s preuzetim vrijednostima. Njihova kasnija promjena ne mijenja izvornu n-torku.

Raspakiravanje se često koristi u petlji nad n-torkama. Svaki zapis može sadržavati naziv i bodove:

```
rezultati = [("Ana", 82), ("Ivo", 74), ("Mia", 91)]

for ime, bodovi in rezultati:
    print(f"{ime}: {bodovi}")
```

```
Ana: 82
Ivo: 74
Mia: 91
```

Vanjska struktura ovdje je lista jer očekujemo da se broj zapisa može mijenjati. Svaki pojedinačni zapis n-torka je s točno dva položaja.

Petlja u svakoj iteraciji preuzima jednu n-torku i odmah je raspakirava. Za prvi zapis varijable dobivaju vrijednosti `"Ana"` i `82`, zatim `"Ivo"` i `74`. Zbog toga svaki zapis mora imati jednak broj elemenata. N-torka s tri vrijednosti ne bi se mogla raspakirati u samo dvije varijable.

5.4 Pretvorba između liste i n-torke

Ponekad podatke prikupimo u listi jer se njihov broj mijenja, a nakon završetka prikupljanja želimo nepromjenjiv zapis. U drugom slučaju primimo n-torku, ali nam je potrebna radna lista u kojoj ćemo promijeniti element. Pretvorba omogućuje stvaranje nove strukture s istim elementima.

Funkcija `tuple()` pretvara prikladnu zbirku u n-torku, a `list()` u listu:

```
vrijednosti = [10, 20, 30]
fiksne_vrijednosti = tuple(vrijednosti)
ponovno_lista = list(fiksne_vrijednosti)

print(fiksne_vrijednosti)
print(ponovno_lista)
```

(10, 20, 30)

[10, 20, 30]

Pretvorba ne znači da su elementi postali trajno zaštićeni u svim varijablama. Stvorena n-torka ne može se mijenjati, ali izvorna lista i dalje postoji i može se mijenjati neovisno.

Funkcije stvaraju novu vanjsku strukturu. Nakon `fiksne_vrijednosti = tuple(vrijednosti)` dodavanje elementa izvornoj listi ne dodaje ga u već stvorenu n-torku. Obrnuto, promjena liste `ponovno_lista` ne mijenja n-torku iz koje je napravljena.

Pregled novih zapisa i funkcija

Zapis ili funkcija	Namjena
<code>(a, b, c)</code>	Stvara n-torku s navedenim elementima
<code>()</code>	Stvara praznu n-torku
<code>(vrijednost,)</code>	Stvara n-torku s jednim elementom
<code>a, b = ntorka</code>	Raspakirava elemente u zasebne varijable
<code>tuple(zbirka)</code>	Stvara novu n-torku iz elemenata zbirke
<code>list(zbirka)</code>	Stvara novu listu iz elemenata zbirke

Zadatak

Zadan je zapis proizvoda `('P-104', 'Tipkovnica', 39.90)`. Raspakirajte ga u tri varijable, ispišite oznaku, naziv i cijenu oblikovanu na dvije decimale te provjerite sadrži li zapis tekst `'Tipkovnica'`. Zatim napravite listu s istim elementima i promijenite cijenu na `34.90`.

▼ Prikaži moguće rješenje i ispis

```
proizvod = ("P-104", "Tipkovnica", 39.90)
oznaka, naziv, cijena = proizvod

print("Oznaka:", oznaka)
print("Naziv:", naziv)
print(f"Cijena: {cijena:.2f} EUR")
print("Sadrži naziv:", "Tipkovnica" in proizvod)

promjenjivi_proizvod = list(proizvod)
promjenjivi_proizvod[2] = 34.90
print(promjenjivi_proizvod)
```

Oznaka: P-104
Naziv: Tipkovnica
Cijena: 39.90 EUR
Sadrži naziv: True
['P-104', 'Tipkovnica', 34.9]

6 Skupovi

Liste dobro čuvaju redoslijed i ponavljanja, ali ta svojstva ponekad nisu dio pitanja koje postavljamo. Ako želimo znati koje su se različite oznake pojavile, koliko je jedinstvenih korisnika pristupilo sustavu ili koji su studenti zajednički dvjema grupama, ponovljene vrijednosti samo otežavaju obradu.

Skup predstavlja pripadnost grupi. Za element je važno nalazi li se u skupu, a ne na kojem je položaju ni koliko je puta prije dodan. Zbog toga je skup prirodan za uklanjanje duplikata, provjere pripadnosti i usporedbu dviju grupa podataka.

6.1 Uklanjanje ponavljanja i provjera jedinstvenih vrijednosti

Pretpostavimo da su oznake kolegija prikupljene iz više izvora. Neke se pojavljuju više puta, ali nas zanima samo koje različite oznake postoje:

Lista u sljedećem programu mora za svaku novu oznaku provjeriti sve što je već spremljeno. Osim toga, programer mora zapamtiti izvršiti tu provjeru na svakom mjestu na kojem se oznaka dodaje. Struktura sama ne sprječava duplikate.

```
oznake = ["INF1", "MAT1", "INF1", "POD1", "MAT1", "INF1"]
jedinstvene_oznake = []

for oznaka in oznake:
    if oznaka not in jedinstvene_oznake:
        jedinstvene_oznake.append(oznaka)

print(jedinstvene_oznake)
```

Takav program radi, ali ručno održava pravilo da se vrijednost smije pojaviti samo jednom. Skup je struktura kojoj je upravo jedinstvenost elemenata osnovno svojstvo.

Pretvorbom liste u skup ne „brišemo pogrešne podatke”, nego mijenjamo pitanje na koje struktura odgovara. Lista govori koje su oznake zabilježene redom, uključujući ponavljanja. Skup govori koje su se različite oznake pojavile. Ako su nam potrebne obje informacije, sačuvati ćemo i izvornu listu i izvedeni skup.

```
oznake = ["INF1", "MAT1", "INF1", "POD1", "MAT1", "INF1"]
jedinstvene_oznake = set(oznake)

print(jedinstvene_oznake)
print("Broj različitih oznaka:", len(jedinstvene_oznake))
```

▼ Prikaži rezultat i važnu napomenu

Skup sadrži tri vrijednosti, ali redoslijed njegova prikaza ne treba unaprijed pretpostaviti. Jedno moguće izvođenje prikazuje:

```
{'INF1', 'MAT1', 'POD1'}
Broj različitih oznaka: 3
```

Drugo izvođenje može iste elemente prikazati drugim redoslijedom. Ako nam je potreban određen redoslijed, rezultat možemo predati funkciji `sorted()`.

`sorted()` ne pretvara skup u „sortirani skup”. Funkcija vraća novu listu čiji su elementi poredani. Izvorni skup i dalje nema položaje na koje se možemo osloniti. To je prikladno kada skup koristimo za jedinstvenost, a listu samo za predvidljiv prikaz rezultata.

💡 Skup

Skup (`set`) je promjenjiva zbirka jedinstvenih elemenata. Ne dopušta duplikate i ne koristimo ga kada je položaj elementa važan. Posebno je koristan za provjeru pripadnosti te usporedbu grupa vrijednosti.

6.2 Stvaranje praznog i popunjenog skupa

Popunjeni skup možemo zapisati vitičastim zagradama. Redoslijed u zapisu nije obećanje redoslijeda kasnijeg ispisa, a jednaku vrijednost dovoljno je navesti jednom:

```
grupe = {"A", "B", "C"}
print(grupe)
```

Prazan skup stvaramo funkcijom `set()`:

```
prazan_skup = set()
prazan_rjecnik = {}

print(type(prazan_skup))
print(type(prazan_rjecnik))
```

```
<class 'set'>
<class 'dict'>
```

⚠️ Prazne vitičaste zagrade nisu skup

Zapis `{}` stvara prazan rječnik. Za prazan skup uvijek koristite `set()`. Popunjeni zapisi razlikuju se po dvotočki: `{"A", "B"}` je skup, a `{"A": 1, "B": 2}` rječnik.

Skup nema indeksiranje ni rezanje. Izraz `grupe[0]` uzrokuje `TypeError` jer skup ne određuje „prvi” element na način na koji to čine lista i n-torka. Kroz elemente možemo prolaziti petljom, ali ne trebamo se oslanjati na njihov redoslijed.

Prazan skup koristimo kada na početku još ne znamo koje će se različite vrijednosti pojaviti. Primjerice, tijekom prolaska kroz popis studenata možemo u skup dodavati oznaku grupe svakog studenta. Ako se ista oznaka doda više puta, u skupu će i dalje postojati samo jednom. Nakon završetka petlje skup zato sadrži sve različite grupe pronađene u podacima. Dodavanje elemenata metodom `add()` detaljnije ćemo prikazati u sljedećem poglavlju.

Skup kao zbirka jest promjenjiv: možemo mu dodavati i uklanjati elemente. Međutim, vrijednost koju spremamo kao jedan element skupa mora biti nepromjenjiva. Broj ili tekst mogu biti elementi skupa, ali lista i rječnik ne mogu jer se njihov sadržaj može naknadno promijeniti. N-torka može biti element skupa ako su i njezini elementi

nepromjenjivi, primjerice `{"A", 1}`. Dakle, možemo mijenjati koje vrijednosti skup sadrži, ali ne možemo u njega spremi promjenjivu vrijednost poput liste.

6.3 Dodavanje i uklanjanje elemenata

Skup je promjenjiv kao cjelina: možemo mu dodavati i uklanjati elemente. Istodobno, svaki pojedinačni element mora biti prikladan za pouzdanu provjeru pripadnosti.

Metoda `add()` dodaje jedan element. Dodavanje vrijednosti koja već postoji ne mijenja skup i ne stvara pogrešku. To ponašanje odgovara značenju skupa: vrijednost ili pripada skupu ili mu ne pripada; nema „dvije kopije” iste pripadnosti.

```
dozvoljene_grupe = {"A", "B"}

dozvoljene_grupe.add("C")
dozvoljene_grupe.add("A")

print(sorted(dozvoljene_grupe))
```

```
['A', 'B', 'C']
```

Metoda `update()` dodaje više elemenata iz druge zbirke:

```
dozvoljene_grupe.update(["D", "E"])
```

Kao kod `extend()` za listu, `update()` prolazi kroz predanu zbirku i dodaje njezine elemente pojedinačno. Razlika je u tome što skup zadržava samo jedinstvene vrijednosti i ne čuva njihov ulazni položaj.

Za uklanjanje elementa prema njegovoj vrijednosti koristimo `remove()` ili `discard()`. Obje metode uklanjaju zadanu vrijednost ako se ona nalazi u skupu. Razlika se pojavljuje kada vrijednost ne postoji: `remove()` tada javlja `KeyError`, dok `discard()` ostavlja skup nepromijenjen i program se nastavlja izvoditi bez pogreške.

```
grupe = {"A", "B", "C"}

grupe.remove("B")
grupe.discard("X")

print(sorted(grupe))
```

```
['A', 'C']
```

`remove()` je koristan kada bi izostanak vrijednosti značio pogrešku u logici programa: očekujemo da element postoji i želimo saznati ako pretpostavka nije točna. `discard()` koristimo kada je prihvatljivo da vrijednost već nije u skupu, primjerice pri čišćenju neobavezne oznake.

6.4 Operacije nad skupovima

Bez skupovnih operacija morali bismo petljom prolaziti kroz jednu grupu, za svaki element provjeravati drugu grupu i ručno graditi rezultat. Skupovi omogućuju da takvu namjeru zapišemo izravno: zajednički članovi, svi članovi ili članovi koji pripadaju samo jednoj strani.

Pretpostavimo da imamo oznake studenata prijavljenih na dvije radionice. Svaka skupovna operacija odgovara na drukčije pitanje, pa operator odabiremo prema odnosu između grupa koji želimo ispitati.

- Operator `|` daje uniju, odnosno sve vrijednosti koje se nalaze u barem jednom skupu.
- Operator `&` daje presjek, odnosno zajedničke vrijednosti.
- Operator `-` daje razliku: vrijednosti iz lijevog skupa kojih nema u desnom.
- Operator `^` daje simetričnu razliku: vrijednosti koje pripadaju točno jednom skupu.

Razlika nije simetrična. `radionica_a - radionica_b` traži članove samo prve radionice, dok bi `radionica_b - radionica_a` tražio članove samo druge. Kod ostalih prikazanih operacija zamjena lijevog i desnog skupa ne mijenja rezultat.

Sljedeći primjer primjenjuje sve četiri operacije na istim skupovima:

```
radionica_a = {"S01", "S02", "S03", "S04"}
radionica_b = {"S03", "S04", "S05"}

print("Barem jedna radionica:", sorted(radionica_a | radionica_b))
print("Obje radionice:", sorted(radionica_a & radionica_b))
print("Samo radionica A:", sorted(radionica_a - radionica_b))
print("Točno jedna radionica:", sorted(radionica_a ^ radionica_b))
```

▼ Prikaži očekivani ispis

```
Barem jedna radionica: ['S01', 'S02', 'S03', 'S04', 'S05']
Obje radionice: ['S03', 'S04']
Samo radionica A: ['S01', 'S02']
Točno jedna radionica: ['S01', 'S02', 'S05']
```

Operacija	Operator	Pitanje na koje odgovara
Unija	<code>a b</code>	Koje se vrijednosti nalaze u barem jednom skupu?
Presjek	<code>a & b</code>	Koje se vrijednosti nalaze u oba skupa?
Razlika	<code>a - b</code>	Koje su vrijednosti u prvom, ali ne i u drugom skupu?

Operacija	Operator	Pitanje na koje odgovara
Simetrična razlika	<code>a ^ b</code>	Koje se vrijednosti nalaze u točno jednom skupu?

Pregled novih zapisa, metoda i operatora

Zapis, metoda ili operator	Namjena
<code>{a, b, c}</code>	Stvara popunjeni skup
<code>set()</code> ili <code>set(zbirka)</code>	Stvara prazan skup ili skup jedinstvenih elemenata zbirke
<code>add(vrijednost)</code>	Dodaje jedan element u skup
<code>update(zbirka)</code>	Dodaje sve elemente druge zbirke u skup
<code>discard(vrijednost)</code>	Uklanja element ako postoji, bez pogreške ako ne postoji
<code>a b</code>	Vraća uniju skupova
<code>a & b</code>	Vraća presjek skupova
<code>a - b</code>	Vraća razliku skupova
<code>a ^ b</code>	Vraća simetričnu razliku skupova

Zadatak

Za dvije grupe polaznika zadani su skupovi `grupa_a = {'Ana', 'Ivo', 'Mia', 'Luka'}` i `grupa_b = {'Mia', 'Luka', 'Ena'}`. Ispišite abecedno sortirane polaznike koji se pojavljuju u barem jednoj grupi, u obje grupe, samo u grupi A i u točno jednoj grupi. Zatim dodajte `Nina` u grupu B i sigurno uklonite `Marko`, ali tako da se ne dogodi greška ako vrijednost ne postoji u skupu.

▼ Prikaži moguće rješenje

```
grupa_a = {"Ana", "Ivo", "Mia", "Luka"}
grupa_b = {"Mia", "Luka", "Ena"}

print("Barem jedna:", sorted(grupa_a | grupa_b))
print("Objje:", sorted(grupa_a & grupa_b))
print("Samo A:", sorted(grupa_a - grupa_b))
print("Točno jedna:", sorted(grupa_a ^ grupa_b))

grupa_b.add("Nina")
grupa_b.discard("Marko")
print("Nova grupa B:", sorted(grupa_b))
```

Barem jedna: ['Ana', 'Ena', 'Ivo', 'Luka', 'Mia']

Obje: ['Luka', 'Mia']

Samo A: ['Ana', 'Ivo']

Točno jedna: ['Ana', 'Ena', 'Ivo']

Nova grupa B: ['Ena', 'Luka', 'Mia', 'Nina']

7 Rječnici

Pretpostavimo da za svakog studenta želimo spremiti broj bodova. Bez strukture koja izravno povezuje ime i rezultat mogli bismo napraviti dvije liste: jednu s imenima i drugu s bodovima. Vezu između podataka predstavljao bi njihov položaj, pa bi bodovi studenta na indeksu `0` morali biti na indeksu `0` druge liste.

Ako želimo pronaći koliko bodova ima Mia, najprije moramo pronaći njezino ime, preuzeti pripadajući indeks i tim indeksom pristupiti drugoj listi:

```
imena = ["Ana", "Ivo", "Mia"]
bodovi = [82, 74, 91]
trazeno_ime = "Mia"

for indeks, ime in enumerate(imena):
    if ime == trazeno_ime:
        print(trazeno_ime, "- broj bodova:", bodovi[indeks])
```

▼ Prikaži očekivani ispis i problem ovakvog pristupa

Mia – broj bodova: 91

Program daje točan rezultat, ali samo dok su obje liste potpuno usklađene. Ako ime dodamo, uklonimo ili premjestimo bez jednake promjene u listi bodova, položaji više neće opisivati iste studente. Python pritom ne mora prijaviti pogrešku, program može povezati studenta s pogrešnim brojem bodova.

Problem nije samo u mogućnosti pogreške. Pitanje „koliko bodova ima Mia?” prirodno polazi od imena, ali program najprije mora tražiti njegov položaj. Potrebna nam je struktura u kojoj ime može izravno voditi do pripadajućeg rezultata.

Rječnik sprema vezu između ključa i vrijednosti. Ključ može biti oznaka studenta, šifra proizvoda ili naziv svojstva poput `"bodovi"`. Time program ne traži „treći podatak”, nego podatak koji je izravno imenovan. Rječnik je zato važan i za pojedinačne zapise i za povezivanje velikog broja oznaka s pripadajućim vrijednostima.

7.1 Pristup podatku prema ključu

Rječnik omogućuje da ime postane ključ prema kojem izravno dohvaćamo bodove. Umjesto dvije odvojene liste i traženja indeksa, svako ime povezujemo s pripadajućom vrijednošću:

```
bodovi = {
    "Ana": 82,
    "Ivo": 74,
    "Mia": 91
}

print(bodovi["Ana"])
print(bodovi["Mia"])
```

82

91

Umjesto položaja koristimo ključ koji opisuje podatak. Par `"Ana": 82` sastoji se od ključa `"Ana"` i vrijednosti `82`.

Ključ mora biti jedinstven unutar jednog rječnika jer prema njemu mora postojati jednoznačan odgovor.

Vrijednosti se smiju ponavljati: Ana i Ivo mogu oboje imati `82` boda. Rječnik čuva redoslijed dodavanja parova, ali glavna svrha ključa nije položaj nego identifikacija vrijednosti.

💡 Rječnik, ključ i vrijednost

Rječnik (`dict`) je promjenjiva zbirka parova ključ-vrijednost. Svaki ključ mora biti jedinstven, a prema ključu pristupamo pripadajućoj vrijednosti.

Ključ ima ulogu oznake podatka. **Vrijednosti** se mogu ponavljati i mogu biti različitih tipova.

Rječnik zapisujemo vitičastim zagradama. Ključ i vrijednost razdvajamo dvotočkom, a parove zarezima. Prazan rječnik zapisujemo kao `{}` ili stvaramo funkcijom `dict()`.

Kao podsjetnik, skup i rječnik koriste vitičaste zagrade, ali rječnik unutar njih ima parove odvojene dvotočkom. Zapis `{"A", "B"}` skup je, `{"A": 82, "B": 74}` rječnik, a prazan zapis `{}` uvijek je rječnik. Prazan skup zapisujemo kao `set()`.

7.2 Dohvaćanje vrijednosti i nepostojeći ključ

Nakon stvaranja rječnika najčešća je operacija dohvaćanje vrijednosti. Uglate zagrade koristimo kada je ključ obavezan dio zapisa i njegov bi izostanak značio da podaci nisu u očekivanom obliku:

```
student = {
    "ime": "Ana",
    "grupa": "A",
    "bodovi": 82,
    "prisutan": True
}

print(student["ime"])
print(student["bodovi"])
```

Ana

82

Sada smo vidjeli dvije različite primjene rječnika. U rječniku `bodovi` ključevi `"Ana"`, `"Ivo"` i `"Mia"` identificiraju studente, a njihove su vrijednosti rezultati. U rječniku `student` ključevi `"ime"`, `"grupa"`, `"bodovi"` i `"prisutan"` predstavljaju svojstva jednoga studenta. Oba zapisa koriste isti tip podataka, ali značenje ključeva određuje uloga rječnika u programu.

Pokušaj `student["email"]` uzrokuje `KeyError` jer ključ ne postoji. Kada je izostanak ključa očekivana mogućnost, možemo najprije provjeriti pripadnost ili koristiti metodu `get()`:

⚠ Operator in kod rječnika provjerava ključeve

Izraz `"ime" in student` provjerava postoji li ključ `"ime"`. Ne provjerava pojavljuje li se tekst među vrijednostima. Za provjeru vrijednosti koristili bismo `"Ana" in student.values()`.

Razlika između tih pristupa opisuje naše očekivanje, a ne samo način izbjegavanja pogreške. Ako svaki student mora imati ključ `"bodovi"`, `KeyError` nam otkriva neispravan zapis. Ako je e-mail neobavezan, nedostajući ključ nije pogreška i `get()` omogućuje dogovoreni zamjenski rezultat.

```
print(student.get("email"))
print(student.get("email", "Nije naveden"))
```

None

Nije naveden

Bez drugog argumenta `get()` za nepostojeći ključ vraća `None`. Drugi argument određuje zamjensku vrijednost. Metoda ne dodaje nedostajući ključ u rječnik, samo određuje rezultat dohvaćanja.

Zamjenska vrijednost treba odgovarati kasnijoj obradi. Tekst `"Nije naveden"` prikladan je za ispis, ali ne i za brojači izračun. Kod brojača često koristimo `0`, dok za podatak bez dogovorene zamjene možemo zadržati `None` i posebno ga provjeriti.

7.3 Dodavanje i promjena parova

Rječnik je promjenjiv, ali za razliku od liste ne moramo odlučivati između `append()` i promjene indeksa. Isti oblik dodjele ima dva ponašanja koja ovise o postojanju ključa: postojeći ključ dobiva novu vrijednost, a novi ključ stvara novi par.

To ponašanje odgovara stvarnim zapisima. Ispravak bodova mijenja postojeće svojstvo, dok naknadno dodavanje grupe proširuje zapis novim svojstvom:

```
student = {"ime": "Ana", "bodovi": 82}

student["bodovi"] = 87
student["grupa"] = "A"

print(student)
```

```
{'ime': 'Ana', 'bodovi': 87, 'grupa': 'A'}
```

Python ne dopušta dva odvojena para s jednakim ključem. Nova dodjela ne stvara drugi ključ `"bodovi"`, nego mijenja vrijednost postojećega.

Zbog toga dodjelu prema ključu trebamo koristiti pažljivo. Tipfeler u nazivu, primjerice `"bodvo"`, ne bi promijenio `"bodovi"`, nego bi stvorio novi ključ. Program bi tada sadržavao oba zapisa. Dosljedni nazivi ključeva važni su jednako kao dosljedni nazivi varijabli.

Metoda `update()` može dodati ili promijeniti više parova odjednom:

```
student.update({"bodovi": 90, "aktivan": True})
print(student)
```

```
{'ime': 'Ana', 'bodovi': 90, 'grupa': 'A', 'aktivan': True}
```

Rječnik predan metodi `update()` obrađuje se par po par. Ključ `"bodovi"` već postoji pa dobiva novu vrijednost, dok se `"aktivan"` dodaje. Ostali parovi ostaju nepromijenjeni.

7.4 Uklanjanje para

Kada podatak više ne pripada zapisu, možemo ukloniti cijeli par. Za razliku od postavljanja vrijednosti na `None`, uklanjanje znači da ključ više ne postoji u rječniku. To utječe na provjeru operatorom `in` i na prolazak kroz ključeve.

Metoda `pop()` uklanja par prema ključu i vraća njegovu vrijednost. Time u jednoj operaciji možemo preuzeti stari podatak i ukloniti ga iz zapisa:

```
student = {"ime": "Ana", "bodovi": 82, "napomena": "provjeriti"}

uklonjena_napomena = student.pop("napomena")

print("Uklonjeno:", uklonjena_napomena)
print(student)
```

```
Uklonjeno: provjeriti
{'ime': 'Ana', 'bodovi': 82}
```

Za nepostojeći ključ možemo navesti zamjensku vrijednost i izbjeći `KeyError`:

```
uklonjeno = student.pop("email", None)
```

Metoda `clear()` uklanja sve parove iz rječnika. Kao kod liste, rječnik nakon toga i dalje postoji, ali je prazan.

Ako nam uklonjena vrijednost nije potrebna, rezultat metode `pop()` ne moramo spremiti. Kada ključ možda ne postoji, drugi argument određuje što metoda treba vratiti umjesto pogreške. Takav zapis koristimo samo kada je izostanak ključa prihvatljiv.

7.5 Prolazak kroz ključeve, vrijednosti i parove

Petlja nad listom u svakoj iteraciji daje element. Rječnik za svaki zapis ima dva povezana dijela, pa prije petlje trebamo odlučiti trebamo li ključeve, vrijednosti ili oboje.

Obična petlja nad rječnikom prolazi kroz njegove ključeve. Prema svakom ključu zatim možemo dohvatiti vrijednost:

```
bodovi = {"Ana": 82, "Ivo": 74, "Mia": 91}
```

```
for ime in bodovi:  
    print(ime, bodovi[ime])
```

▼ Prikaži očekivani ispis

```
Ana 82  
Ivo 74  
Mia 91
```

Pomoću metoda `keys()`, `values()` i `items()` možemo izričito odabrati želimo li prolaziti kroz ključeve, vrijednosti ili cijele parove:

```
print("Ključevi:")  
for ime in bodovi.keys():  
    print(ime)  
  
print("Vrijednosti:")  
for rezultat in bodovi.values():  
    print(rezultat)  
  
print("Parovi:")  
for ime, rezultat in bodovi.items():  
    print(f"{ime}: {rezultat}")
```

▼ Prikaži očekivani ispis

```
Ključevi:  
Ana  
Ivo  
Mia  
Vrijednosti:  
82  
74  
91  
Parovi:  
Ana: 82  
Ivo: 74  
Mia: 91
```

Metoda `items()` daje parove koje možemo raspakirati u dvije varijable. Redoslijed je redoslijed dodavanja parova u rječnik, ali ključeve po značenju ne smatramo položajima kao indekse liste.

`keys()` izričito pokazuje da trebamo ključeve, iako bi obična petlja dala isti rezultat. `values()` koristimo kada identitet ključa nije važan, primjerice za zbroj svih bodova. `items()` je najprikladniji za izvještaj koji mora sadržavati i oznaku i pripadajuću vrijednost.

7.6 Brojanje pojavljivanja pomoću rječnika

Pretpostavimo da imamo popis grupa kojima studenti pripadaju i želimo izračunati koliko je studenata u svakoj grupi:

```
grupe = ["A", "B", "A", "C", "B", "A"]
```

Ako unaprijed znamo da postoje samo grupe "A", "B" i "C", mogli bismo napraviti zaseban brojač za svaku grupu:

```
broj_a = 0
broj_b = 0
broj_c = 0

for grupa in grupe:
    if grupa == "A":
        broj_a += 1
    elif grupa == "B":
        broj_b += 1
    elif grupa == "C":
        broj_c += 1

print("A:", broj_a)
print("B:", broj_b)
print("C:", broj_c)
```

▼ Prikaži očekivani ispis i problem ovakvog pristupa

```
A: 3
B: 2
C: 1
```

Program radi za zadane tri grupe, ali je svaka grupa ugrađena u kod. Ako se u podacima pojavi grupa "D", program je neće prebrojiti dok ne dodamo novu varijablu, novu granu `elif` i novu naredbu za ispis. Što je više mogućih kategorija, to program sadrži više gotovo jednakih brojača i uvjeta.

Rječnik omogućuje da oznaka grupe bude ključ, a njezin broj pojavljivanja vrijednost. Umjesto unaprijed pripremljenih varijabli `broj_a`, `broj_b` i `broj_c`, počinjemo praznim rječnikom. Kada petlja pronade novu oznaku, program za nju u rječniku stvara novi par. Kada oznaku pronade ponovno, povećava njezinu postojeću vrijednost. Isti kod tako može prebrojiti i kategorije koje nismo unaprijed naveli:

```
grupe = ["A", "B", "A", "C", "B", "A"]
broj_po_grupi = {}

for grupa in grupe:
    trenutni_broj = broj_po_grupi.get(grupa, 0)
    broj_po_grupi[grupa] = trenutni_broj + 1

print(broj_po_grupi)
```

▼ Prikaži ispis i objašnjenje

```
{'A': 3, 'B': 2, 'C': 1}
```

Pri prvom pojavljivanju grupe ključ još ne postoji, pa `get(grupa, 0)` vraća početnu vrijednost `0`. Nakon povećavanja u rječnik se sprema `1`. Pri sljedećem pojavljivanju istog ključa `get()` vraća postojeći broj, koji zatim povećavamo.

Tijek za prva tri elementa izgleda ovako: nakon prve oznake `"A"` rječnik je `{"A": 1}`; nakon `"B"` postaje `{"A": 1, "B": 1}`; nakon druge oznake `"A"` vrijednost uz ključ `"A"` povećava se na `2`. Novi ključ stvaramo samo pri prvom pojavljivanju kategorije.

Isti se postupak može kraće zapisati:

```
for grupa in grupe:
    broj_po_grupi[grupa] = broj_po_grupi.get(grupa, 0) + 1
```

Kraća verzija nije drukčiji algoritam, samo spaja dohvaćanje, povećavanje i dodjelu u jednu naredbu.

7.7 Dopušteni tipovi ključeva

Rječnik mora moći pouzdano pronaći istu vrijednost svaki put kada dobije jednak ključ. Kada bi se sadržaj ključa mogao promijeniti nakon umetanja, rječnik više ne bi mogao jamčiti gdje se pripadajuća vrijednost nalazi. Zato ključ mora biti nepromjenjiva vrijednost.

Česti su ključevi tekstovi, brojevi i n-torke sastavljene od nepromjenjivih elemenata. N-torka je korisna kada jedan ključ prirodno čini kombinacija više dijelova, primjerice godina, mjesec i dan:

```
temperature = {
    (2026, 10, 1): 18.5,
    (2026, 10, 2): 19.0
}

print(temperature[(2026, 10, 2)])
```

Lista ne može biti ključ jer se njezin sadržaj može promijeniti. Pokušaj `{[2026, 10, 1]: 18.5}` završava pogreškom `TypeError: unhashable type: 'list'`.

Vrijednost rječnika nema isto ograničenje. Ona može biti broj, tekst, lista, skup, drugi rječnik ili druga prikladna vrijednost. Ograničenje se odnosi na ključ jer se upravo prema njemu podatak pronalazi.

Pregled novih zapisa i metoda

Zapis ili metoda	Namjena
<code>{kljuc: vrijednost}</code>	Stvara rječnik s navedenim parovima
<code>{}</code>	Stvara prazan rječnik
<code>rjecnik[kljuc]</code>	Dohvaća vrijednost i javlja <code>KeyError</code> ako ključ ne postoji
<code>rjecnik[kljuc] = vrijednost</code>	Dodaje novi par ili mijenja vrijednost postojećeg ključa
<code>get(kljuc, zadano=None)</code>	Dohvaća vrijednost bez pogreške za nepostojeći ključ
<code>keys()</code>	Daje ključeve rječnika
<code>values()</code>	Daje vrijednosti rječnika
<code>items()</code>	Daje parove ključeva i vrijednosti

Zadatak

Zadan je sljedeći rječnik proizvoda:

```
proizvod = {  
    "oznaka": "P-104",  
    "naziv": "Tipkovnica",  
    "cijena": 39.90,  
    "na_stanju": 12  
}
```

Ispišite naziv i cijenu proizvoda, povećajte stanje za 5, dodajte ključ `kategorija` s vrijednošću `"Oprema"`, sigurno dohvatite nepostojeći ključ `dobavljac` uz tekst `"Nije naveden"` te na kraju ispišite sve parove u obliku `ključ: vrijednost`.

▼ Prikaži moguće rješenje i ispis

```
proizvod = {
    "oznaka": "P-104",
    "naziv": "Tipkovnica",
    "cijena": 39.90,
    "na_stanju": 12
}

print("Naziv:", proizvod["naziv"])
print(f"Cijena: {proizvod['cijena']:.2f} EUR")

proizvod["na_stanju"] += 5
proizvod["kategorija"] = "Oprema"

print("Dobavljač:", proizvod.get("dobavljac", "Nije naveden"))

for kljuc, vrijednost in proizvod.items():
    print(f"{kljuc}: {vrijednost}")
```

Naziv: Tipkovnica
Cijena: 39.90 EUR
Dobavljač: Nije naveden
oznaka: P-104
naziv: Tipkovnica
cijena: 39.9
na_stanju: 17
kategorija: Oprema

8 Ugniježdene strukture podataka

8.1 Zašto jedan zapis često sadrži više vrijednosti

Dosad smo uglavnom radili sa zbirkama u kojima su svi elementi bili jednostavne vrijednosti. Primjerice, lista bodova sadržavala je samo brojeve, a skup grupa samo tekstualne oznake. Takva je organizacija dovoljna dok svaki element opisuje samo jednu osobinu.

U stvarnim podacima jedan zapis najčešće sadrži više povezanih vrijednosti. Jedan student može imati ime, grupu, broj bodova i podatak o predanom radu. Jedan proizvod može imati oznaku, naziv, cijenu i stanje. Te vrijednosti ne smijemo promatrati kao nepovezane podatke jer zajedno opisuju isti objekt.

Mogli bismo napraviti zasebnu listu imena, zasebnu listu grupa i zasebnu listu bodova, ali tada bi povezanost ovisila o položaju. Prvi element svake liste morao bi pripadati istom studentu, drugi element drugom studentu i tako dalje. Dodavanje ili uklanjanje vrijednosti samo iz jedne liste pomaknulo bi podatke i proizvelo pogrešne zapise bez nužne Pythonove pogreške.

Pouzdanije je sve podatke jednoga studenta povezati u jednu strukturu, a zatim više takvih zapisa spremati u drugu strukturu. Time nastaju dvije razine:

1. Zbirku svih zapisa
2. Vrijednosti koje pripadaju jednom zapisu

Vanjska struktura odgovara na pitanje koji zapis obrađujemo, a unutarnja koji podatak toga zapisa trebamo. Struktura koja sadrži druge strukture naziva se ugniježdena struktura. Pri pristupu podacima čitamo je izvana prema unutra: najprije odabiremo zapis, a zatim podatak unutar njega.

Ugniježdena struktura

Ugniježdena struktura nastaje kada je element jednog kontejnera drugi kontejner. Primjeri su lista n-torki, lista rječnika, rječnik lista i rječnik rječnika.

8.2 Lista n-torki

Ako svaki zapis ima jednak, mali broj vrijednosti i njihov je redoslijed unaprijed dogovoren, možemo koristiti listu n-torki. Lista nam omogućuje da dodajemo i obrađujemo više zapisa, dok n-torka drži vrijednosti jednoga zapisa na stalnim položajima.

U sljedećem primjeru svaki zapis ima tri položaja: ime, grupu i bodove. Dogovor o značenju položaja mora vrijediti za svaku n-torku u listi:

```
rezultati = [  
    ("Ana", "A", 82),  
    ("Ivo", "B", 74),  
    ("Mia", "A", 91)  
]  
  
print(rezultati[0])  
print(rezultati[0][2])
```

```
('Ana', 'A', 82)  
82
```

Izraz `rezultati[0]` dohvaća prvu n-torku. Dobivena vrijednost je `("Ana", "A", 82)`. Tek se na toj vrijednosti primjenjuje sljedeći indeks `[2]`, koji dohvaća treći element n-torke, odnosno bodove. Zato izraz `rezultati[0][2]` čitamo slijeva nadesno: prvi zapis, zatim treća vrijednost toga zapisa.

Takav je zapis kratak, ali iz samog izraza `[0][2]` nije vidljivo što broj `2` predstavlja. Programer mora znati dogovoreni raspored svih položaja. Lista n-torki zato je praktična za male i stabilne zapise, ali postaje teža za čitanje kada zapis ima puno svojstava.

Petlja s raspakiravanjem poboljšava čitljivost jer položajima odmah daje smislene nazive:

```
for ime, grupa, bodovi in rezultati:  
    if bodovi >= 60:  
        status = "prolaz"  
    else:  
        status = "nije prolaz"  
  
    print(f"{ime}, grupa {grupa}: {bodovi} – {status}")
```

U svakoj iteraciji petlja uzima jednu n-torku i raspakirava je u varijable `ime`, `grupa` i `bodovi`. Broj varijabli mora odgovarati broju elemenata u svakoj n-torci. Nakon raspakiravanja više ne moramo pisati indekse, pa je uvjet `bodovi >= 60` jasniji od izraza koji bi više puta koristio položaj `[2]`.

8.3 Lista rječnika

Kada zapis ima više svojstava ili želimo da njihovo značenje bude vidljivo u kodu, lista rječnika često je čitljivija. Vanjska lista i dalje predstavlja zbirku studenata, ali je svaki student opisan rječnikom. Umjesto pamćenja da su bodovi na trećem položaju, dohvaćamo ih smislenim ključem `"bodovi"`.

Ovakav je zapis nešto dulji, ali se može razumjeti bez poznavanja posebnog dogovora o položajima:

```
studenti = [
    {"ime": "Ana", "grupa": "A", "bodovi": 82},
    {"ime": "Ivo", "grupa": "B", "bodovi": 74},
    {"ime": "Mia", "grupa": "A", "bodovi": 91}
]

print(studenti[0])
print(studenti[0]["bodovi"])
```

```
{'ime': 'Ana', 'grupa': 'A', 'bodovi': 82}
82
```

Prvi indeks odabire studenta, a ključ `"bodovi"` podatak tog studenta. Izraz `studenti[0]["bodovi"]` zato znači: iz liste dohvati prvi zapis, a iz toga rječnika vrijednost povezanu s ključem `"bodovi"`. Zapis je dulji od n-torke, ali se njegovo značenje vidi iz samog izraza.

Rječnici u listi obično koriste jednake ključeve kako bi ih isti dio programa mogao jednako obraditi. Python to ne zahtijeva, pa odgovornost za dosljednost ostaje na nama. Ako jedan zapis koristi ključ `"bodovi"`, a drugi `"rezultat"`, ista naredba neće raditi za oba zapisa.

```
for student in studenti:
    if student["bodovi"] >= 60:
        status = "prolaz"
    else:
        status = "nije prolaz"

    print(f'{student["ime"]}: {status}')
```

▼ Prikaži očekivani ispis

```
Ana: prolaz
Ivo: prolaz
Mia: prolaz
```

U svakoj iteraciji varijabla `student` predstavlja jedan cijeli rječnik, a ne samo ime ili broj bodova. Iz njega prema potrebi dohvaćamo pojedina svojstva. Ključevi su jednaki u svim zapisima, pa isti kod može obraditi svakog studenta bez zasebnih varijabli i ponavljanja naredbi.

8.4 Promjena podatka u ugniježđenoj strukturi

Ugniježđenost ne znači da se podaci više ne mogu mijenjati. Lista i rječnik promjenjivi su, pa možemo doći do točno određenog zapisa i u njemu promijeniti samo jedno svojstvo. Pritom ponovno pratimo razine strukture: najprije odabiremo element vanjske liste, a zatim ključ unutarnjeg rječnika.

Pretpostavimo da je naknadnom provjerom ustanovljeno da Ivo ima `79`, a ne `74` boda. Promjena treba zahvatiti samo njegov zapis:

```
studenti[1]["bodovi"] = 79
studenti[1]["napomena"] = "ispravljen unos"

print(studenti[1])
```

```
{'ime': 'Ivo', 'grupa': 'B', 'bodovi': 79, 'napomena': 'ispravljen unos'}
```

Najprije `studenti[1]` pronalazi drugi rječnik, a zatim `["bodovi"]` u tom rječniku pronalazi vrijednost koju treba zamijeniti. Vanjska lista ostaje ista zbirka, a ostali studentski zapisi nisu promijenjeni.

Dodjela prema novom ključu `"napomena"` dodaje novi par samo tom studentu. Zbog toga svi rječnici u listi više nemaju potpuno jednake ključeve. To je dopušteno, ali pri kasnijem čitanju neobaveznog podatka treba koristiti `get()` ili prethodno provjeriti postoji li ključ. Za podatke koji moraju postojati u svakom zapisu dosljednost ključeva trebamo održavati od samog stvaranja zapisa.

8.5 Rječnik čije su vrijednosti liste

Isti skup stvarnih podataka možemo organizirati na više načina. Smjer organizacije ovisi o pitanju koje često postavljamo i podatku prema kojem želimo brzo doći do određene skupine vrijednosti.

Ako rezultate najčešće obrađujemo student po student, lista rječnika prirodan je izbor. Ako često pitamo „koji bodovi pripadaju grupi A?“, korisnije je da grupa bude ključ, a svi njezini rezultati vrijednost u obliku liste. Tada ne moramo prolaziti kroz sve studentske zapise kako bismo prikupili bodove jedne grupe:

```
bodovi_po_grupi = {
    "A": [82, 91, 76],
    "B": [74, 68]
}

for grupa, bodovi in bodovi_po_grupi.items():
    prosjek = sum(bodovi) / len(bodovi)
    print(f"Grupa {grupa}: {prosjek:.2f}")
```

Grupa A: 83.00

Grupa B: 71.00

Rječnik omogućuje izravan odabir grupe prema ključu, a pripadajuća lista čuva sve rezultate te grupe. U petlji `items()` istodobno daje oznaku grupe i njezinu listu bodova, pa se prosjek može računati za svaku grupu istim postupkom.

Ova organizacija, međutim, više ne sadrži identitet pojedinog studenta. Ako su nam kasnije potrebna imena ili drugi podaci, morali bismo proširiti unutarnje zapise. Ne postoji jedna obavezna struktura za sve programe, odabir ovisi o tome koje veze moramo sačuvati i koje obrade program treba izvesti.

Pregled zapisa ugniježđenih struktura

Zapis	Namjena
<code>[(a, b), (c, d)]</code>	Lista n-torki s jednakim rasporedom položaja

Zapis**Namjena**`{"kljuc": vrijednost}, ...]`

Lista zapisa čijim svojstvima pristupamo prema ključu

`struktura[indeks][kljuc]`

Dohvaća zapis iz liste, a zatim vrijednost iz rječnika

`for a, b in lista_ntorki:`

Raspakirava svaku n-torku tijekom prolaska

`{kljuc: [a, b, c]}`

Rječnik koji uz svaki ključ čuva listu vrijednosti

i Zadatak

Zadana je lista rječnika s proizvodima. Ispišite nazive svih proizvoda na stanju, izračunajte ukupnu vrijednost zalihe kao zbroj umnožaka cijene i količine te napravite skup svih različitih kategorija.

```
proizvodi = [  
    {"naziv": "Tipkovnica", "kategorija": "Oprema", "cijena": 39.90, "kolicina": 4},  
    {"naziv": "Miš", "kategorija": "Oprema", "cijena": 18.50, "kolicina": 0},  
    {"naziv": "Bilježnica", "kategorija": "Pribor", "cijena": 3.20, "kolicina": 15}  
]
```

▼ Prikaži moguće rješenje i ispis

```
proizvodi = [  
    {"naziv": "Tipkovnica", "kategorija": "Oprema", "cijena": 39.90, "kolicina": 4},  
    {"naziv": "Miš", "kategorija": "Oprema", "cijena": 18.50, "kolicina": 0},  
    {"naziv": "Bilježnica", "kategorija": "Pribor", "cijena": 3.20, "kolicina": 15}  
]  
  
ukupna_vrijednost = 0  
kategorije = set()  
  
for proizvod in proizvodi:  
    if proizvod["kolicina"] > 0:  
        print("Na stanju:", proizvod["naziv"])  
  
        ukupna_vrijednost += proizvod["cijena"] * proizvod["kolicina"]  
        kategorije.add(proizvod["kategorija"])  
  
print(f"Ukupna vrijednost: {ukupna_vrijednost:.2f} EUR")  
print("Kategorije:", sorted(kategorije))
```

Na stanju: Tipkovnica

Na stanju: Bilježnica

Ukupna vrijednost: 207.60 EUR

Kategorije: ['Oprema', 'Pribor']

9 Sažetak složenih tipova podataka

Lista, n-torka, skup i rječnik mogu sadržavati više vrijednosti, ali ih ne organiziraju prema jednakim pravilima. Odgovarajuću strukturu odabiremo prema tome je li važan redoslijed, smiju li se vrijednosti ponavljati, trebaju li se podaci mijenjati i pristupamo li im prema položaju ili ključu.

9.1 Usporedba obrađenih struktura

Tip	Primjer zapisa	Redoslijed	Može se mijenjati	Ponavljanje vrijednosti	Način pristupa
Lista (<code>list</code>)	<code>[72, 88, 65]</code>	Da	Da	Da	Prema indeksu
N-torka (<code>tuple</code>)	<code>(72, 88, 65)</code>	Da	Ne	Da	Prema indeksu
Skup (<code>set</code>)	<code>{72, 88, 65}</code>	Ne treba pretpostavljati	Da	Ne	Prema vrijednosti, bez indeksa
Rječnik (<code>dict</code>)	<code>{"Ana": 72, "Ivo": 88}</code>	Čuva redoslijed dodavanja	Da	Ključevi ne, vrijednosti da	Prema ključu

Redoslijed, promjenjivost i ponavljanje nisu samo tehničke razlike. Određuju odgovara li struktura značenju podataka. Primjerice, skup nije lošija lista bez redoslijeda, nego struktura za problem u kojem su jedinstvenost i pripadnost važnije od položaja. Rječnik ne zamjenjuje indeks drugim nazivom, nego izravno povezuje ključ s pripadajućom vrijednošću.

9.2 Namjena pojedinih struktura

Isti podaci mogu se spremati u različite strukture, ali se time mijenja način na koji ih koristimo. Ako oznake spremimo u listu, čuvamo njihov redoslijed i ponavljanja. Ako ih pretvorimo u skup, olakšavamo provjeru jedinstvenosti, ali gubimo ponavljanja i ne pristupamo im prema indeksu. Struktura zato nije samo drukčiji zapis iste stvari; ona izražava što nam je u tim podacima važno.

Lista čuva niz elemenata kojima pristupamo prema položaju. Korisna je za podatke koji se tijekom rada dopunjuju ili ispravljaju, poput rezultata mjerenja ili popisa prijavljenih studenata. Ponavljanja ostaju sačuvana: dva jednaka rezultata i dalje predstavljaju dva odvojena mjerenja. Redoslijed elemenata pritom ne znači da su oni sortirani.

N-torka također čuva redoslijed i omogućuje pristup indeksima, ali nakon stvaranja ne dopušta zamjenu, dodavanje ni uklanjanje svojih elemenata. Često predstavlja zapis s dogovorenim položajima, poput koordinata točke ili datuma zapisanog kao godina, mjesec i dan. Značenje položaja određuje program: n-torka sama ne zna da prvi broj predstavlja godinu ili da koordinate trebaju imati dva dijela.

Skup čuva jedinstvene vrijednosti. Koristan je kada nas zanima koje su se različite oznake pojavile, pripada li neka vrijednost određenoj grupi ili koji su elementi zajednički dvjema grupama. Ponavljanja i položaji nisu sačuvani, pa iz skupa ne možemo zaključiti koliko se puta vrijednost pojavila ni kojim je redom unesena.

Skupovne operacije omogućuju uniju, presjek, razliku i simetričnu razliku.

Rječnik povezuje ključeve s vrijednostima. Ključ može označavati svojstvo jednoga zapisa, primjerice "bodovi", ili identificirati podatak, primjerice šifru proizvoda uz koju je spremljena cijena. Ključevi su jedinstveni, dok se vrijednosti smiju ponavljati. Dodjela prema postojećem ključu mijenja njegovu vrijednost, a prema novom ključu dodaje novi par.

9.3 Povezivanje struktura

Ugniježdene strukture povezuju ta svojstva na više razina. U listi rječnika vanjska lista čuva više zapisa, a svaki unutarnji rječnik opisuje svojstva jednoga zapisa. U rječniku lista ključ može predstavljati grupu, a pripadajuća lista sve njezine rezultate. Svaka razina zadržava pravila svojega tipa: zapis iz liste dohvaćamo indeksom, a svojstvo iz rječnika ključem.

U istom programu često koristimo više struktura za različite dijelove obrade. Lista prijava može sačuvati svaku prijavu i njihov redoslijed, skup izdvojiti sve različite grupe, a rječnik povezati svaku grupu s brojem prijavljenih studenata. Te strukture predstavljaju različite informacije dobivene iz istih početnih podataka.

Pretvorba između struktura može promijeniti informacije koje ostaju dostupne. Pretvaranjem liste u skup gubimo ponavljanja i izvorni redoslijed. Naknadno pretvaranje toga skupa u listu ne vraća izgubljene podatke. Zato izvorna zbirka i izvedene strukture mogu istodobno imati važnu ulogu u programu.

10 Pogreške pri radu sa složenim tipovima

Kod jednostavne varijable obično pratimo jednu vrijednost i njezin tip. Kod složenih struktura moramo pratiti još nekoliko pitanja: postoji li traženi element, pristupamo li mu indeksom ili ključem, podržava li struktura odabranu metodu i mijenjamo li slučajno istu strukturu kroz koju prolazimo.

Poruka o pogrešci zato nije samo obavijest da program ne radi. Vrsta pogreške i redak na kojem se pojavila pomažu utvrditi koja pretpostavka o strukturi nije bila točna. Umjesto nasumičnog mijenjanja koda, prvo trebamo pročitati izraz slijeva nadesno i odrediti tip vrijednosti na kojoj je operacija pokušana.

10.1 Pogrešan indeks, ključ ili vrijednost

Različite strukture prijavljuju različite pogreške kada traženi podatak ne postoji. Lista i n-torka očekuju valjan položaj, rječnik postojeći ključ, a metode `index()` i `remove()` vrijednost koja se doista nalazi u zbirci. Naziv pogreške često otkriva koju smo pretpostavku prekršili.

Postupak	Primjer	Pogreška
Pristup nepostojećem indeksu	<code>[10, 20][2]</code>	<code>IndexError</code>
Pristup nepostojećem ključu	<code>{"a": 1}["b"]</code>	<code>KeyError</code>
Traženje nepostojeće vrijednosti	<code>[10, 20].index(30)</code>	<code>ValueError</code>
Uklanjanje nepostojeće vrijednosti liste	<code>[10, 20].remove(30)</code>	<code>ValueError</code>
Promjena n-torke	<code>(10, 20)[0] = 5</code>	<code>TypeError</code>
Indeksiranje skupa	<code>{10, 20}[0]</code>	<code>TypeError</code>

Primjerice, `IndexError` ne znači da je broj kao tip podatka neispravan, nego da zadani položaj ne postoji u toj listi ili n-torci. `KeyError` govori da rječnik nema traženi ključ. `TypeError` u ovim primjerima znači da odabrana operacija nije dopuštena za taj tip strukture.

Pogrešku ne trebamo ukloniti nasumičnim mijenjanjem koda. Najprije treba provjeriti odgovara li operacija strukturi i je li nepostojanje podatka očekivan slučaj. Ako podatak prema pravilima programa mora postojati, pogreška nam može korisno otkriti neispravne ulazne podatke. Ako je njegov izostanak dopušten, program treba predvidjeti tu mogućnost.

- Prije pristupa indeksu možemo provjeriti duljinu liste i utvrditi postoji li traženi položaj
- Prije `remove()` ili `index()` možemo provjeriti pripadnost operatorom `in` ako je izostanak vrijednosti dopušten
- Za neobavezan ključ rječnika možemo koristiti `get()` i odabrati smislenu zamjensku vrijednost
- Za uklanjanje iz skupa bez pogreške kada element ne postoji možemo koristiti `discard()`

Nije potrebno unaprijed spriječiti svaku moguću pogrešku. Provjera treba odgovarati pravilima problema. Ako svaki studentski zapis mora imati ključ `"bodovi"`, korištenje `get()` s vrijednošću `0` moglo bi prikriti neispravan zapis i proizvesti pogrešan rezultat. U tom je slučaju `KeyError` korisniji od tihog nastavka programa.

10.2 Metoda pripada određenom tipu

Različite strukture mogu sadržavati jednake elemente, ali zbog toga ne podržavaju jednake operacije. Lista, n-torka i skup u sljedećem primjeru svi sadrže brojeve 1, 2 i 3, no predstavljaju različita pravila o redoslijedu, ponavljanju i promjenama:

```
vrijednosti_lista = [1, 2, 3]
vrijednosti_ntorka = (1, 2, 3)
vrijednosti_skup = {1, 2, 3}
```

`append()` pripada listi jer dodaje jedan element na njezin kraj. Skup nema pojam kraja, pa za dodavanje koristi `add()`. N-torka nema metodu za dodavanje jer je nepromjenjiva. Ako pozovemo metodu koju tip nema, Python javlja `AttributeError`.

Zato odabir metode ne počinje pitanjem „kako se zvala naredba za dodavanje?“, nego pitanjem „koji je tip ove vrijednosti i kakvu promjenu želim napraviti?“. Funkcija `type()` i prethodni ispis međurezultata mogu pomoći kada tip nije očit iz većeg programa.

⚠ Provjerite tip prije odabira metode

Slična namjena ne znači isti naziv metode. Listi dodajemo element metodom `append()`, skupu metodom `add()`, a rječniku dodjelom prema ključu ili metodom `update()`.

10.3 Promjena zbirke tijekom prolaska

Petlja očekuje da će prolaziti kroz elemente određene zbirke. Ako iz te iste zbirke tijekom prolaska uklanjamo elemente, položaji preostalih elemenata mijenjaju se dok petlja već napreduje. Zbog toga neki elementi mogu ostati neprovjereni ili Python može prijaviti pogrešku.

Sljedeći program pokušava uklanjati parne brojeve iz iste liste koju obrađuje:

```
brojevi = [1, 2, 3, 4, 5, 6]

for broj in brojevi:
    if broj % 2 == 0:
        brojevi.remove(broj)

print(brojevi)
```

Za ovu listu rezultat slučajno može izgledati očekivano, ali postupak nije pouzdan za sve rasporede vrijednosti. Nakon uklanjanja broja 2, broj 3 pomiče se na njegovo staro mjesto, dok petlja prelazi na sljedeći položaj. Time se pomaknuta vrijednost može preskočiti. Program koji ponekad daje dobar rezultat i dalje može sadržavati logičku pogrešku.

Jasnije je proći kroz izvornu listu bez njezina mijenjanja i napraviti novu listu s elementima koje želimo zadržati:

```

brojevi = [1, 2, 3, 4, 5, 6]
neparni = []

for broj in brojevi:
    if broj % 2 != 0:
        neparni.append(broj)

print(neparni)

```

[1, 3, 5]

Izvorna lista tada ostaje stabilna tijekom cijele petlje, a rezultat gradimo odvojeno. Kod rječnika i skupa promjena broja elemenata tijekom petlje može izazvati `RuntimeError`. Zato najprije možemo prikupiti ključeve ili vrijednosti koje treba ukloniti pa ih ukloniti nakon petlje, ili možemo stvoriti novu strukturu s elementima koje želimo zadržati.

10.4 Sustavna provjera ugniježdene strukture

Dugačak izraz u ugniježđenoj strukturi može sadržavati više pristupa. Ako `studenti[1]["bodovi"]` ne radi, nije dovoljno znati da je „problem negdje u podacima”. Trebamo utvrditi na kojoj razini stvarna vrijednost odstupa od očekivane.

Izraz možemo privremeno rastaviti na korake. Najprije provjerimo vanjsku strukturu i indeks, a zatim unutarnju vrijednost i ključ:

1. Je li `studenti` lista?
2. Postoji li indeks `1`?
3. Je li vrijednost na tom indeksu rječnik?
4. Postoji li ključ `"bodovi"`?
5. Je li dohvaćena vrijednost tipa koji očekujemo?

Ispis međurezultata pomaže otkriti na kojoj je razini pretpostavka pogrešna:

```

print(type(studenti))
print(studenti[1])
print(type(studenti[1]))
print(studenti[1].keys())

```

Ako `studenti[1]` ispiše tekst umjesto rječnika, problem nije u ključu `"bodovi"`, nego u sadržaju liste. Ako ispiše rječnik bez toga ključa, problem je u strukturi konkretnog zapisa. Takvo postupno sužavanje problema učinkovitije je od mijenjanja cijelog izraza odjednom.

Ne moramo sve ispisne naredbe zadržati u konačnom programu. Koristimo ih tijekom provjere, a nakon pronalaska pogreške uklanjamo ili pretvaramo u komentare.

11 Pregled struktura, funkcija i metoda

Sljedeće tablice okupljaju zapise korištene u ovoj vježbi kako ih ne biste morali tražiti kroz sva prethodna poglavlja. Služe kao podsjetnik tijekom rješavanja zadataka, a ne kao popis koji treba učiti napamet.

Najprije odredite što podatak predstavlja i koju promjenu ili provjeru trebate izvesti. Tek zatim u tablici pronađite zapis odgovarajuće operacije. Ako samo odaberemo metodu s poznatim nazivom bez provjere tipa i namjene, lako možemo primijeniti ispravnu sintaksu na pogrešnu strukturu.

11.1 Osnovni zapisi i operatori

Pregled osnovnih zapisa

Zapis	Namjena
<code>[a, b, c]</code>	Stvara listu
<code>(a, b, c)</code>	Stvara n-torku
<code>(a,)</code>	Stvara n-torku s jednim elementom
<code>{a, b, c}</code>	Stvara popunjeni skup
<code>set()</code>	Stvara prazan skup
<code>{kljuc: vrijednost}</code>	Stvara rječnik
<code>struktura[indeks]</code>	Dohvaća element liste ili n-torke prema položaju
<code>rjecnik[kljuc]</code>	Dohvaća vrijednost prema ključu
<code>vrijednost in struktura</code>	Provjerava pripadnost; kod rječnika provjerava ključeve
<code>a b</code>	Unija skupova
<code>a & b</code>	Presjek skupova
<code>a - b</code>	Razlika skupova
<code>a ^ b</code>	Simetrična razlika skupova

11.2 Funkcije

Funkcije korištene sa složenim tipovima

Funkcija	Rezultat
<code>dict()</code>	Novi prazan rječnik ili rječnik iz prikladnih parova
<code>enumerate(zbirka, start=0)</code>	Parovi indeksa i vrijednosti za prolazak petljom

Funkcija	Rezultat
<code>len(zbirka)</code>	Broj elemenata
<code>list(zbirka)</code>	Nova lista iz elemenata zbirke
<code>max(zbirka)</code>	Najveći element neprazne zbirke
<code>min(zbirka)</code>	Najmanji element neprazne zbirke
<code>set(zbirka)</code>	Novi skup jedinstvenih elemenata
<code>sorted(zbirka, reverse=False)</code>	Nova sortirana lista
<code>sum(zbirka)</code>	Zbroj brojčanih elemenata
<code>tuple(zbirka)</code>	Nova n-torka iz elemenata zbirke
<code>type(vrijednost)</code>	Tip vrijednosti

11.3 Metode

Metode lista, skupova i rječnika

Tip	Metoda	Namjena
Lista	<code>append(vrijednost)</code>	Dodaje jedan element na kraj
Lista i rječnik	<code>clear()</code>	Uklanja sve elemente ili parove
Lista	<code>copy()</code>	Stvara plitku kopiju liste
Lista i n-torka	<code>count(vrijednost)</code>	Broji pojavljivanja vrijednosti
Lista	<code>extend(zbirka)</code>	Dodaje sve elemente druge zbirke
Lista i n-torka	<code>index(vrijednost)</code>	Vraća indeks prvog pojavljivanja
Lista	<code>insert(indeks, vrijednost)</code>	Umeće element na zadani položaj
Lista i rječnik	<code>pop(oznaka)</code>	Uklanja i vraća element; lista koristi indeks, a rječnik ključ
Lista i skup	<code>remove(vrijednost)</code>	Uklanja vrijednost; javlja pogrešku ako ona ne postoji
Lista	<code>sort(reverse=False)</code>	Sortira postojeću listu
Skup	<code>add(vrijednost)</code>	Dodaje jedan element
Skup	<code>discard(vrijednost)</code>	Uklanja element ako postoji

Tip	Metoda	Namjena
Skup i rječnik	<code>update(zbirka)</code>	Skupu dodaje elemente, a rječniku dodaje ili mijenja parove
Rječnik	<code>get(kljuc, zadano=None)</code>	Dohvaća vrijednost bez <code>KeyError</code>
Rječnik	<code>items()</code>	Daje parove ključeva i vrijednosti
Rječnik	<code>keys()</code>	Daje ključeve
Rječnik	<code>values()</code>	Daje vrijednosti

12 Samostalni zadaci

Zadatke riješite u zasebnim Python datotekama. Najprije sami odaberite odgovarajuću strukturu, funkcije i metode. Rješenja se nalaze u posljednjem poglavlju i služe za provjeru nakon pokušaja.

12.1 Zadatak 1

Zadana je lista temperatura `[18.2, 21.5, 19.8, 23.1, 20.4]`. Ispišite broj mjerenja, prvo i posljednje mjerenje, srednja tri mjerenja te provjerite nalazi li se vrijednost `19.8` u listi.

12.2 Zadatak 2

Zadana je lista naziva `['analiza.csv', 'izvjestaj.pdf', 'podaci.xlsx']`. Dodajte `graf.png` na kraj, umetnite `README.txt` na početak, uklonite `izvjestaj.pdf` prema vrijednosti i ispišite konačnu listu.

12.3 Zadatak 3

Zadana je lista bodova `[55, 72, 88, 49, 91, 63]`. Napravite novu listu koja sadrži samo rezultate s najmanje `60` bodova. Izvornu listu nemojte mijenjati. Ispišite obje liste i prosjek prolaznih rezultata.

12.4 Zadatak 4

Zadana je lista vremena `[34, 28, 41, 28, 37, 31]`. Ispišite najmanje i najveće vrijeme, broj pojavljivanja vrijednosti `28`, indeks njezina prvog pojavljivanja te novu listu sortiranu silazno. Izvorni redoslijed mora ostati sačuvan.

12.5 Zadatak 5

Napravite listu `izvorno = [10, 20, 30]` i njezinu zasebnu kopiju. U kopiji promijenite drugi element u `200`, a zatim ispišite obje liste kako biste pokazali da izvorna lista nije promijenjena.

12.6 Zadatak 6

Zadana je n-torka `('HR', 'Pula', 52100)`. Raspakirajte je u varijable za državu, grad i poštanski broj te ispišite rečenicu `Pula (HR) ima poštanski broj 52100.`. Zatim provjerite sadrži li n-torka vrijednost `'Pula'`.

12.7 Zadatak 7

Objasnite i programom provjerite razliku između zapisa `(7)` i `(7,)`. Ispišite tip obje vrijednosti i duljinu one koja predstavlja n-torku.

12.8 Zadatak 8

Zadana je lista oznaka `['A1', 'B2', 'A1', 'C3', 'B2', 'D4']`. Napravite skup jedinstvenih oznaka, ispišite ih sortirano i ispišite broj različitih oznaka.

12.9 Zadatak 9

Za skupove `prijavljeni = {'S01', 'S02', 'S03', 'S04'}` i `prisutni = {'S02', 'S04', 'S05'}` ispišite studente koji su prijavljeni i prisutni, prijavljeni ali nisu prisutni te sve oznake koje se pojavljuju u barem jednom skupu.

12.10 Zadatak 10

Napravite rječnik knjige s ključevima `naslov`, `autor`, `godina` i `dostupna`. Ispišite naslov i godinu, promijenite dostupnost, dodajte ključ `broj_stranica` te sigurno dohvatite ključ `isbn` uz zamjensku poruku `Nije naveden`.

12.11 Zadatak 11

Zadana je lista kategorija `['A', 'B', 'A', 'C', 'B', 'A', 'C', 'C']`. Napravite rječnik koji za svaku kategoriju sadrži broj pojavljivanja i ispišite sve parove.

12.12 Zadatak 12

Zadan je rječnik `cijene = {'olovka': 1.20, 'biljeznica': 3.50, 'mapa': 2.80}`. Povećajte svaku cijenu za `10 %` i ispišite naziv proizvoda te novu cijenu na dvije decimale. Ne mijenjajte broj parova u rječniku tijekom petlje.

12.13 Zadatak 13

Zadana je lista n-torki `[('Ana', 82), ('Ivo', 57), ('Mia', 91), ('Luka', 64)]`. Raspakiravanjem u petlji ispišite imena i bodove studenata koji su ostvarili najmanje `60` bodova te izračunajte broj takvih studenata.

12.14 Zadatak 14

Zadana je lista rječnika:

```
mjerenja = [  
    {"stanica": "Pula", "temperatura": 22.4},  
    {"stanica": "Pazin", "temperatura": 19.8},  
    {"stanica": "Pula", "temperatura": 23.1},  
    {"stanica": "Rovinj", "temperatura": 21.7}  
]
```

Ispišite sva mjerenja za Pulu, napravite skup različitih stanica i izračunajte prosjek svih temperatura.

12.15 Zadatak 15

Zadana je lista prijava:

```
prijave = [  
    {"oznaka": "S01", "radionica": "Python"},  
    {"oznaka": "S02", "radionica": "Podaci"},  
    {"oznaka": "S01", "radionica": "Python"},  
    {"oznaka": "S03", "radionica": "Python"},  
    {"oznaka": "S04", "radionica": "Podaci"}  
]
```

Odbacite potpuno jednake ponovljene prijave, ispišite jedinstvene prihvaćene prijave te izradite rječnik s brojem prijava po radionici. Ista oznaka smije biti prijavljena na različite radionice.

13 Rješenja samostalnih zadataka

13.1 Rješenje zadatka 1

▼ Prikaži moguće rješenje i ispis

```
temperature = [18.2, 21.5, 19.8, 23.1, 20.4]

print("Broj mjerenja:", len(temperature))
print("Prvo:", temperature[0])
print("Posljednje:", temperature[-1])
print("Srednja tri:", temperature[1:4])
print("Postoji 19.8:", 19.8 in temperature)
```

Broj mjerenja: 5
Prvo: 18.2
Posljednje: 20.4
Srednja tri: [21.5, 19.8, 23.1]
Postoji 19.8: True

13.2 Rješenje zadatka 2

▼ Prikaži moguće rješenje i ispis

```
nazivi = ["analiza.csv", "izvjestaj.pdf", "podaci.xlsx"]

nazivi.append("graf.png")
nazivi.insert(0, "README.txt")
nazivi.remove("izvjestaj.pdf")

print(nazivi)
```

['README.txt', 'analiza.csv', 'podaci.xlsx', 'graf.png']

13.3 Rješenje zadatka 3

▼ Prikaži moguće rješenje i ispis

```
bodovi = [55, 72, 88, 49, 91, 63]
prolazni = []

for rezultat in bodovi:
    if rezultat >= 60:
        prolazni.append(rezultat)

prosijek = sum(prolazni) / len(prolazni)

print("Izvorno:", bodovi)
print("Prolazni:", prolazni)
print("Prosijek:", prosijek)
```

Izvorno: [55, 72, 88, 49, 91, 63]
Prolazni: [72, 88, 91, 63]
Prosijek: 78.5

13.4 Rješenje zadatka 4

▼ Prikaži moguće rješenje i ispis

```
vremena = [34, 28, 41, 28, 37, 31]
silazno = sorted(vremena, reverse=True)

print("Najmanje:", min(vremena))
print("Najveće:", max(vremena))
print("Broj vrijednosti 28:", vremena.count(28))
print("Prvi indeks vrijednosti 28:", vremena.index(28))
print("Silazno:", silazno)
print("Izvorno:", vremena)
```

Najmanje: 28
Najveće: 41
Broj vrijednosti 28: 2
Prvi indeks vrijednosti 28: 1
Silazno: [41, 37, 34, 31, 28, 28]
Izvorno: [34, 28, 41, 28, 37, 31]

13.5 Rješenje zadatka 5

▼ Prikaži moguće rješenje i ispis

```
izvorno = [10, 20, 30]
kopija = izvorno.copy()

kopija[1] = 200

print("Izvorno:", izvorno)
print("Kopija:", kopija)
```

Izvorno: [10, 20, 30]
Kopija: [10, 200, 30]

13.6 Rješenje zadatka 6

▼ Prikaži moguće rješenje i ispis

```
lokacija = ("HR", "Pula", 52100)
drzava, grad, postanski_broj = lokacija

print(f"{grad} ({drzava}) ima poštanski broj {postanski_broj}.")
print("Sadrži Pulu:", "Pula" in lokacija)
```

Pula (HR) ima poštanski broj 52100.
Sadrži Pulu: True

13.7 Rješenje zadatka 7

▼ Prikaži moguće rješenje i objašnjenje

```
broj = (7)
ntorka = (7,)

print(type(broj))
print(type(ntorka))
print(len(ntorka))
```

```
<class 'int'>
<class 'tuple'>
1
```

Obične zagrade samo grupiraju izraz, pa (7) ostaje cijeli broj. Zarez u (7,) stvara n-torku s jednim elementom.

13.8 Rješenje zadatka 8

▼ Prikaži moguće rješenje i ispis

```
oznake = ["A1", "B2", "A1", "C3", "B2", "D4"]
jedinstvene = set(oznake)

print("Oznake:", sorted(jedinstvene))
print("Broj oznaka:", len(jedinstvene))
```

Oznake: ['A1', 'B2', 'C3', 'D4']
Broj oznaka: 4

13.9 Rješenje zadatka 9

▼ Prikaži moguće rješenje i ispis

```
prijavljeni = {"S01", "S02", "S03", "S04"}
prisutni = {"S02", "S04", "S05"}

print("Prijavljeni i prisutni:", sorted(prijavljeni & prisutni))
print("Prijavljeni, nisu prisutni:", sorted(prijavljeni - prisutni))
print("Barem jedan skup:", sorted(prijavljeni | prisutni))
```

Prijavljeni i prisutni: ['S02', 'S04']
Prijavljeni, nisu prisutni: ['S01', 'S03']
Barem jedan skup: ['S01', 'S02', 'S03', 'S04', 'S05']

13.10 Rješenje zadatka 10

▼ Prikaži moguće rješenje i ispis

```
knjiga = {
    "naslov": "Podaci u praksi",
    "autor": "Ana Anić",
    "godina": 2025,
    "dostupna": True
}

print("Naslov:", knjiga["naslov"])
print("Godina:", knjiga["godina"])

knjiga["dostupna"] = False
knjiga["broj_stranica"] = 240

print("ISBN:", knjiga.get("isbn", "Nije naveden"))
print(knjiga)
```

Naslov: Podaci u praksi

Godina: 2025

ISBN: Nije naveden

{'naslov': 'Podaci u praksi', 'autor': 'Ana Anić', 'godina': 2025, 'dostupna': False, 'broj_stranica': 240}

13.11 Rješenje zadatka 11

▼ Prikaži moguće rješenje i ispis

```
kategorije = ["A", "B", "A", "C", "B", "A", "C", "C"]
broj_pojavljivanja = {}

for kategorija in kategorije:
    broj_pojavljivanja[kategorija] = broj_pojavljivanja.get(kategorija, 0) + 1

for kategorija, broj in broj_pojavljivanja.items():
    print(f"{kategorija}: {broj}")
```

A: 3

B: 2

C: 3

13.12 Rješenje zadatka 12

▼ Prikaži moguće rješenje i ispis

```
cijene = {"olovka": 1.20, "biljeznica": 3.50, "mapa": 2.80}
```

```
for proizvod in cijene:  
    cijene[proizvod] *= 1.10
```

```
for proizvod, cijena in cijene.items():  
    print(f"{proizvod}: {cijena:.2f} EUR")
```

olovka: 1.32 EUR

biljeznica: 3.85 EUR

mapa: 3.08 EUR

Promjena vrijednosti postojećeg ključa dopuštena je tijekom petlje jer broj ključeva ostaje jednak. Dodavanje ili uklanjanje ključeva tijekom istog prolaska moglo bi izazvati `RuntimeError`.

13.13 Rješenje zadatka 13

▼ Prikaži moguće rješenje i ispis

```
rezultati = [("Ana", 82), ("Ivo", 57), ("Mia", 91), ("Luka", 64)]  
broj_prolaznih = 0
```

```
for ime, bodovi in rezultati:  
    if bodovi >= 60:  
        print(f"{ime}: {bodovi}")  
        broj_prolaznih += 1  
  
print("Broj prolaznih:", broj_prolaznih)
```

Ana: 82

Mia: 91

Luka: 64

Broj prolaznih: 3

13.14 Rješenje zadatka 14

▼ Prikaži moguće rješenje i ispis

```
mjerenja = [  
    {"stanica": "Pula", "temperatura": 22.4},  
    {"stanica": "Pazin", "temperatura": 19.8},  
    {"stanica": "Pula", "temperatura": 23.1},  
    {"stanica": "Rovinj", "temperatura": 21.7}  
]  
  
stanice = set()  
zbroj = 0  
  
for mjerjenje in mjerenja:  
    if mjerjenje["stanica"] == "Pula":  
        print("Pula:", mjerjenje["temperatura"])  
  
        stanice.add(mjerjenje["stanica"])  
        zbroj += mjerjenje["temperatura"]  
  
prosjek = zbroj / len(mjerenja)  
  
print("Stanice:", sorted(stanice))  
print(f"Prosjek: {prosjek:.2f}")
```

```
Pula: 22.4  
Pula: 23.1  
Stanice: ['Pazin', 'Pula', 'Rovinj']  
Prosjek: 21.75
```

13.15 Rješenje zadatka 15

▼ Prikaži moguće rješenje i ispis

```

prijave = [
    {"oznaka": "S01", "radionica": "Python"},
    {"oznaka": "S02", "radionica": "Podaci"},
    {"oznaka": "S01", "radionica": "Python"},
    {"oznaka": "S03", "radionica": "Python"},
    {"oznaka": "S04", "radionica": "Podaci"}
]

videne_prijave = set()
prihvacene = []
broj_po_radionici = {}

for prijava in prijave:
    kljuc_prijave = (prijava["oznaka"], prijava["radionica"])

    if kljuc_prijave in videne_prijave:
        continue

    videne_prijave.add(kljuc_prijave)
    prihvacene.append(prijava)

    radionica = prijava["radionica"]
    broj_po_radionici[radionica] = broj_po_radionici.get(radionica, 0) + 1

print("Prihvaćene prijave:")
for prijava in prihvacene:
    print(f"{prijava['oznaka']}: {prijava['radionica']}")

print("Broj po radionicama:")
for radionica, broj in broj_po_radionici.items():
    print(f"{radionica}: {broj}")

```

Prihvaćene prijave:

S01: Python

S02: Podaci

S03: Python

S04: Podaci

Broj po radionicama:

Python: 2

Podaci: 2

N-torka (**oznaka**, **radionica**) može biti element skupa jer je nepromjenjiva. Time jednaku prijavu prepoznamo kao ponavljanje, ali ista oznaka i druga radionica čine drugi ključ.